

Global Pass Barriers Without Per-Resource RHI Tracking: A Cross-Vendor Study with BLADE

Dzmitry Malyshau

July 2026

Abstract

Explicit graphics APIs expose memory dependencies, per-resource accesses, and image layouts. WGPU reconstructs and validates this state; BLADE keeps Vulkan images in **GENERAL**, tracks no per-resource state, and issues global pass-boundary barriers. We isolate barrier placement and stage/access scope within BLADE, compare matched WGPU programs end-to-end, and measure six GPUs from four vendors, including exploratory Apple/Metal results.

Removing fifteen redundant barriers from sixteen independent compute passes reduces GPU span by 29.3% [29.2, 29.4] on an RTX 5070 and 32.3% [32.1, 32.6] on an RX 7900 XT. It reduces independent-render span by 32.4% and 7.3% on the same discrete parts, but increases it by 42.4% on a Radeon 780M at 32 passes, beyond that count’s stability floor. No dependent-chain placement effect clears the study’s cell-specific stability criterion. Deriving global barrier scope from the pass kinds around each boundary, without tracking any resource, saves 5.0% [4.8, 5.0] on an NVIDIA graphics chain and 6.7% [4.5, 6.9] on an AMD compute chain; no AMD render-involving scope cell resolves. WGPU’s record-and-submit cost is higher in every measured cell, but this end-to-end difference cannot be attributed to tracking alone.

RADV source shows why command counts do not predict these costs: broad global dependencies expand to several flush and invalidate requests that the driver may partly elide. It also shows that persistent **GENERAL** retains DCC under the measured RDNA conditions but disables FMASK. The resulting BLADE direction is lightweight aggregate pass-kind state in a tracking-free RHI: an upstream render graph selects global dependency cuts, while aliasing, cross-queue use, exceptional layouts, and arbitrary per-resource DAG edges remain resource-aware engine responsibilities.

1 Introduction

Explicit GPU APIs expose synchronization so applications can preserve parallelism and avoid unnecessary cache operations. That control has a cost: somebody has to know how every buffer and image subresource is used, maintain that state across command buffers, validate conflicts, and generate transitions. This machinery sits on a

CPU-critical command recording path. The WGPU implementation describes its trackers as some of the hottest code in its codebase [21].

Production engines show both the scale and why this work is normally centralized. Activision reports that the first deployment of its Task Graph contained 236 tasks, 258 resources, and 92 barrier calls with 328 barrier elements; the graph later grew to roughly 350 tasks [3]. Unreal’s Render Dependency Graph likewise uses pass resource declarations to derive subresource transitions, asynchronous compute fences, transient aliasing, validation, and parallel recording [6]. At that scale the useful question is more precise than “track or do not track”: must the low-level graphics abstraction duplicate per-resource knowledge, or can an engine-level graph remain the source of truth and give the abstraction a smaller execution contract?

BLADE [13] tests the latter engineering hypothesis. The distinction is not that its API exposes fewer state objects — WGPU’s does not expose resource states either, and tracking is an internal matter for both — but that BLADE does not maintain the state at all. On Vulkan, ordinary images stay in the **GENERAL** image layout and a broad memory barrier orders writes before later reads and writes at pass boundaries, so nothing has to be remembered between one pass and the next. What surfaces in the API is a consequence rather than the point: no per-resource work at record time, and an application that wants overlap has to say so itself. A render graph and BLADE are therefore composable rather than competing designs: the graph can choose the dependency boundaries while the RHI declines to rediscover them. An application without such a graph can retain BLADE’s conservative automatic boundaries.

The question is newly relevant because `VK_KHR_unified_image_layouts` guarantees, when enabled, that **GENERAL** is as efficient as specialized layouts in most uses [23]. The extension removes one reason for tracking image state, but explicitly leaves barrier granularity as a separate performance question. It is not universal in the measured set: four of the five Vulkan device-driver pairs advertise it (Table 2), and these current 2025–2026 drivers are not a representative deployment sample. This paper therefore measures **GENERAL** on drivers with and without the extension

rather than assuming the guarantee.

The immediate motivation is concrete. A BLADE user reported that their renderer’s planar-reflection pass and main pass, which share no resources, were being serialized on AMD hardware purely because BLADE placed a barrier between them [7]. That report, the fix it produced, and the question of how far it generalizes are the subject of Section 4.

This paper is organized around the following research questions:

RQ1: How does host cost scale with passes and dependency density for tracking-free and tracked record-and-submit paths?

RQ2: When does removing redundant global barriers reduce device span, and when is it neutral or harmful?

RQ3: How do matched BLADE and WGPU implementations differ end-to-end?

RQ4: How do the answers differ between compute and graphics workloads, and across architectures?

The contributions are:

- a cost model that distinguishes tracking, barrier precision, barrier placement, and image layouts without claiming that every factor is independently controlled;
- a controlled within-BLADE evaluation of automatic versus application-declared barrier placement on five Vulkan GPUs, including a negative result where coarse placement wins;
- a barrier that narrows stage and access scope from the kinds of the surrounding passes, keeping the no-resource-state contract, with its correctness argument, its measured effect at fixed placement, and the two-sided misprediction of the driver-source reading about where it would pay;
- matched BLADE and WGPU workloads with matching post-run output hashes, plus captures that compare their emitted barrier requests, for end-to-end comparison;
- a reading of the AMD open-source driver that identifies, in driver operations rather than folklore, what a global barrier and a persistent `GENERAL` layout ask for — and, from the same driver, conditions under which part of that request can be elided;
- practical guidance identifying the workload envelope in which tracking-free synchronization is appropriate.

2 Background

2.1 Execution and memory dependencies

A Vulkan barrier describes both an execution dependency between pipeline stages and a memory dependency between access types. Resource-specific barriers can additionally identify the affected buffer or image range. Narrow scopes may preserve unrelated work, whereas overly broad scopes can drain the pipeline [4, 19]. A global memory barrier is nevertheless attractive when many resources share the same dependency because it combines them in one command and often maps to hardware operations whose granularity is already coarse.

These dimensions must not be collapsed into a single “barrier quality” variable. A system may:

1. place barriers only at true hazards or at every pass boundary;
2. use global or resource-specific memory scopes;
3. use broad or precise pipeline stages and access masks; and
4. transition images between specialized layouts or retain one layout.

This decomposition is not specific to Vulkan. Direct3D 12 Enhanced Barriers independently expose synchronization scope, access scope, and texture layout, and distinguish global, buffer, and texture barriers. A global barrier affects the indicated resource-memory accesses without identifying a resource or changing a texture layout [9]. BLADE’s policy selects the analogous global point and separately chooses a persistent image layout. We do not benchmark Direct3D 12; its API is evidence that the axes are real interface choices, not performance evidence for BLADE.

2.2 What a coarse barrier costs a driver

Vendor guidance states that broad barriers are expensive but rarely says what work they request. The AMD open-source Vulkan driver answers that mechanics question directly, so we read it rather than infer it. All statements below refer to Mesa `main` at revision `d18d598e275d` [15].

RADV converts a dependency into a set of cache-operation flags in `radv_src_access_flush` and `radv_dst_access_flush`. Before either runs, the shared Vulkan runtime expands the access mask [16]: `VK_ACCESS_2_MEMORY_WRITE_BIT` becomes *every* write access reachable from the given stage mask. BLADE’s barrier names `ALL_COMMANDS`, so the expansion is total, and consequently every branch of `radv_src_access_flush` fires:

- `COLOR_ATTACHMENT_WRITE` triggers `FLUSH_AND_INV_CB`, plus colour metadata handling;

- `DEPTH_STENCIL_ATTACHMENT_WRITE` triggers `FLUSH_AND_INV_DB`, plus depth metadata handling;
- `TRANSFER_WRITE` requests both attachment-cache paths and their metadata handling;
- `TRANSFORM_FEEDBACK_WRITE` requests an L2 write-back; and
- command-preprocess writes request an L2 invalidation; shader-storage and acceleration-structure writes do so where the relevant memory is not coherent.

Two details make the global form more pessimistic than an *image*-scoped barrier carrying the same access masks. (A buffer barrier supplies no image either.) First, RADV initializes `has_CB_meta` and `has_DB_meta` to true and only clears them when an image argument is supplied; a global memory barrier therefore always adds `FLUSH_AND_INV_CB_META` and `FLUSH_AND_INV_DB_META` even when no compressed image is involved. The destination side is pessimistic in the same way and for the same reason. Second, the L2-coherence check `radv_image_is_l2_coherent` requires an image, so a global barrier cannot use it; the source comment notes that for memory barriers without an image the state at rest “often devolves to just VRAM/GTT anyway”.

The execution dependency is separate and equally blunt. RADV maps source stages to partial flushes: `ALL_COMMANDS` appears in both the list that sets `CS_PARTIAL_FLUSH` and the list that sets `PS_PARTIAL_FLUSH`. A partial flush waits for all in-flight waves of that type to retire. Placing BLADE’s barrier at every pass boundary therefore requests both compute and pixel partial flushes between every pair of passes, whether or not they share data; either request can be cheap when no matching work is in flight. This source-level mechanism is consistent with the user report in Section 4, but does not by itself establish its cost.

2.2.1 What the same driver takes back

The paragraphs above read the request. The driver’s answer to it is a few lines further down the same function, and we record it here because Section 6.3 needed it and did not have it.

`radv_dst_access_flush` does not use the barrier’s masks alone. It first widens its notion of coherence: `image_is_coherent` is set whenever `can_skip_buffer_l2_flushes` holds and no render-backend-incoherent image is currently dirty. The first condition is a property of the part — GFX9, or GFX10 and later with a coherent TCC — and the second is tracked across the command buffer, set only when a render pass binds an attachment for which `radv_image_is_l2_coherent` is false. On GFX10 and

later that predicate reduces to whether the image is pipe-misaligned.

The consequence is conditional but broad: on the measured RDNA parts, while no pipe-misaligned attachment is dirty, every destination-side `INV_L2` the coarse barrier asks for is dropped. The driver has already made the argument that motivated our narrower barrier, and made it without needing the application to say anything. What a narrower scope can still remove is the source-side flushes and the pixel drain — operations whose cost is proportional to work in flight and to dirty cache state, both of which are absent in exactly the workloads where the narrowing is cleanest to measure. Section 6.3 reports the measurement, including the one component this account does not predict; the explanation here arrived after the numbers, and it is a reading of source rather than a second measurement.

2.3 Image layouts

Vulkan image layouts communicate usage information that may select compression metadata, attachment paths, or transfer representations. Vendor guidance has historically recommended specialized layouts, particularly on AMD [1]. The unified-image-layouts extension reflects a hardware and driver trend in which most layouts are physically identical. It allows `GENERAL` in most positions and promises optimal performance, while retaining exceptions such as initialization from `UNDEFINED`, presentation, and some video interactions [23].

RADV lets us be more specific than the extension proposal, because the layout policy is three readable predicates [15]:

- `radv_layout_dcc_compressed` ends by returning true when the GFX level is at least GFX10 *or* the layout is not `GENERAL`. Delta colour compression therefore survives `GENERAL` on RDNA and later, and is lost on GFX9 and earlier. One earlier branch qualifies this: an image whose queue-family mask includes the compute family loses DCC in `GENERAL` on hardware that cannot compress DCC through image stores. BLADE creates every image `EXCLUSIVE` on one queue family, so the qualification does not apply to anything measured here, but it does apply to an application that shares images with an async-compute queue. Subject to the function’s other transfer-queue and feedback-loop cases, the relevant conclusion is narrower: for the measured single-queue colour attachments on GFX10 and later, `GENERAL` by itself does not disable DCC.
- `radv_layout_is_htile_compressed` returns true for `GENERAL` when TC-compatible HTILE is enabled for that level, the image can be used on the graphics queue, and a driver-configuration override has not disabled the case. The accompanying comment states that it exists specifically to help “apps that use

GENERAL for the main depth pass”, and gives “the image doesn’t have the storage bit set” as part of its justification. That condition is enforced earlier when RADV decides whether the image may use HTILE, rather than retested by this layout predicate.

- `radv_layout_fmash_compression` returns `RADV_FMASH_COMPRESSION_NONE` for `GENERAL` unconditionally, before any of its other cases. A persistent `GENERAL` layout therefore forgoes FMASK compression for multisampled colour; the bandwidth cost is expected from that source reading but is not measured here.

Consequently, this work uses “no layout tracking” to mean no steady-state tracking for ordinary application images. It does not claim that all layout transitions or image-specific synchronization disappear, and it identifies MSAA colour as the case where the claim is weakest.

2.4 WebGPU and wgpu

WebGPU defines usage scopes that reject conflicting uses and provide a safe, portable programming contract [20]. WGPU represents buffer and texture states in dense trackers, merges per-scope usage, and generates transitions as usage scopes enter a command buffer and between submitted command buffers [21]. Its Vulkan backend emits `VkBufferMemoryBarrier` and `VkImageMemoryBarrier` structures whose stages, accesses, and image layouts are derived from the recorded usage. That makes the requests resource-specific; it does not prove minimum stages, minimum calls, or optimal placement. This work does not treat that tracking as accidental overhead: it implements guarantees that BLADE’s unsafe API intentionally does not provide.

The comparison instead asks what that end-to-end design costs, and whether a narrower native contract is useful for applications that already know their dependency structure.

3 Blade Synchronization Model

BLADE exposes transfer, compute, and render passes within a command encoder. It does not attach a current state to resource handles. The Vulkan backend places the following dependency before each pass, and once more when the encoder is finished:

```
srcStage = ALL_COMMANDS
dstStage = ALL_COMMANDS
srcAccess = MEMORY_WRITE
dstAccess = MEMORY_READ | MEMORY_WRITE
```

Images are initialized from `UNDEFINED` to `GENERAL` and remain in `GENERAL` for ordinary rendering, sampling, storage, and copies. Presentation images still transition to the presentation layout. BLADE enables

`VK_KHR_unified_image_layouts` on drivers that advertise it.

Since the resolution of the field report, `CommandEncoderDesc` carries a `manual_barriers` flag [11]. When set, the automatic pass-boundary barrier is suppressed and the application calls `encoder.barrier()` where its dependency graph requires one. The emitted barrier is identical; only the placement changes. This is the mechanism the evaluation controls, and its design history is Section 4.

3.1 Narrowing the barrier without tracking resources

Section 2.2 shows that most of what the barrier above *asks* of RADV comes from the two words `ALL_COMMANDS` and `MEMORY_WRITE`, not from the barrier being global. That suggests a cheaper barrier that still tracks nothing: an encoder always knows what kind of pass it just closed and what kind it is about to open, and a pass kind is one enum per encoder, not a table indexed by resource. The distinction between what a barrier asks for and what it costs is the one this configuration went on to measure, in both directions; we keep the original reasoning here because the configuration is worth having anyway, and report where it held and where it did not in Section 6.3.

We therefore added a second scope, and the first question it raises is how much the application has to say for it to work. The answer is: nothing, for the barriers the encoder places itself. Two quantities are needed, and the encoder already holds both.

The *source* scope must cover every write the barrier has to make available. That is the union of the kinds of the passes recorded since the previous barrier, which the encoder accumulates in one bitmask. Taking the union rather than just the last pass is what makes this safe when the application suppressed some boundaries: whichever passes went unseparated, their writes are still in the set. The *destination* scope is the kind of the pass being opened, and the encoder is inside `compute()`, `render()`, `transfer()`, or `acceleration_structure()` when it places the barrier, so the kind is the method that was called. Nothing is added to the pass declaration, and no resource is consulted.

The one case that is not derivable is an explicitly placed barrier, because the consumer has not been declared when `barrier()` is called. That one takes its scope as an argument, and is emitted where it is written: its source scope narrows like any other, but its destination stays `ALL_COMMANDS`. Holding the request back until the next pass opened would recover the destination as well, and we measured that it does, but a synchronization primitive that does not appear where the application put it is a poor trade for a few percent. The asymmetry is instead reported as a result: it is the measurable cost of not knowing what comes next.

The derivation itself is a table over pass kinds:

Pass kind	Source stage	Source access
transfer	TRANSFER	TRANSFER_WRITE
compute	COMPUTE_SHADER	SHADER_WRITE
render	ALL_GRAPHICS	attachment, SHADER_WRITE
accel. struct.	AS_BUILD TRANSFER	AS_WRITE TRANSFER_WRITE

The destination masks mirror this with the read accesses each kind can perform. The first barrier of an encoder starts with the source set seeded to “unknown”, so it is as wide as before; anything recorded outside the pass model, such as image initialization, adds “unknown” to the set for the same reason. The barrier emitted when the encoder finishes keeps a wide *destination*, since the next queue consumer is unknown, but its source is derived like any other. Host visibility separately relies on queue completion and the memory’s host-coherency or invalidation rules.

Correctness rests on the standard availability-and-visibility argument. An automatically placed barrier names every producer accumulated since the previous barrier on its source side and the pass being opened on its destination side. An explicit barrier names the same accumulated producers and keeps an `ALL_COMMANDS` destination, which covers any later pass. Thus every write ordered by the policy is made available by a source scope that includes its producing pass kind and visible to a destination scope that includes its consumer.

Synchronization validation was necessary rather than decorative here. It found that Blade applies two driver workarounds which add transfer accesses to every barrier; those are legal alongside `ALL_COMMANDS` and illegal alongside a narrowed stage mask, and they are also unnecessary in the narrowed case, because a narrowed scope names the accesses of the passes that actually ran. During development, the final implementation reported no Khronos synchronization hazards or validation errors across all thirty-six workload and policy combinations, and every policy produced the same output hash. The output hashes are retained with the timing collections; the development-time synchronization-validation logs are not. The validation result is therefore not yet archival evidence and must be repeated and published before submission.

On RADV the difference is concrete. A compute-to-compute boundary drops from {compute and pixel partial flush, colour-block flush, depth-block flush, both meta-data flushes, L2 invalidate, L2 write-back} to {compute partial flush, L2 invalidate}. A render-to-compute boundary keeps the colour and depth flushes, which are real, but loses the compute drain. What it does *not* drop is anything on the destination’s cache side, because on these parts the driver had already dropped it (Section 2.2.1) — the first hint that what this configuration removes and what it would save were not going to line up.

The evaluated contract is deliberately limited:

- synchronization within one Vulkan queue and command stream;

- explicit host and cross-queue synchronization outside the pass model;
- explicit initialization and presentation transitions; and
- native validation during correctness testing, but no safe-API guarantee in production.

The Metal backend is a different animal and should not be described as tracking-free. Metal tracks hazards for resources created in the default `MTLHazardTrackingModeTracked` mode [2], and BLADE relies on that: `manual_barriers` has no effect there, and the backend issues no inter-pass synchronization of its own. On Metal the tracking has moved into the framework, not disappeared.

4 Field Report: Barrier-Induced Serialization on AMD

The controlled experiments in this paper were prompted by a user report [7]. A renderer built on BLADE drew a planar reflection and a main view into separate colour and depth targets, resolved each from MSAA, and then drew reflective and refractive geometry that depended on both. Conceptually the reflection and main passes are independent. In practice BLADE emitted this sequence:

```
full_barrier(); raytrace()
full_barrier(); reflection_depth_prepass()
pipeline_barrier(); render_reflection()
full_barrier(); resolve_reflection_msaa()
full_barrier(); main_depth_prepass()
pipeline_barrier(); render_main()
full_barrier(); resolve_main_msaa()
full_barrier(); render_reflective_refractive()
```

Figure 1 draws that sequence against the dependency graph it came from. The reporter observed in a GPU profiler that the barriers forced the AMD scheduler to run the reflection work to completion before starting the main view, and asked whether the hardware could overlap them at all if the barriers were removed. Section 2.2 answers the mechanism question: on RADV each `full_barrier` sets both `CS_PARTIAL_FLUSH` and `PS_PARTIAL_FLUSH`. The execution dependency therefore does not depend on the driver noticing that the two passes are unrelated, although the latency of either flush still depends on matching work being in flight. The driver cannot discard the dependency merely because it sees no shared resource: the application requested `ALL_COMMANDS` ordering.

The reporter prototyped a flag to disable automatic barriers, removed the barriers between the ray-tracing, reflection, and main passes, kept one before the dependent refraction pass, and reported that the ray-tracing and reflection passes then executed concurrently. Two

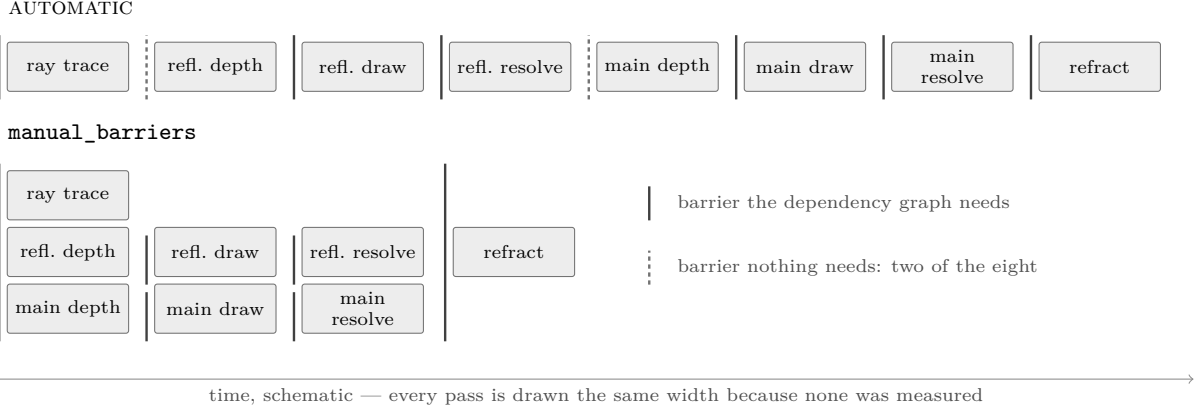


Figure 1: The field report’s frame, as BLADE emitted it and as it was restructured. The figure shows what the barriers *permit*, not what the hardware achieved. Two of the eight barriers separate work with no resource in common — ray tracing from the reflection view, and the reflection view from the main one — and on RADV each requests both compute and pixel partial flushes without regard to whether the passes share a resource (Section 2.2). Removing those two is what let the reporter’s profiler show ray tracing and reflection running together. The remaining six are real edges in the graph and stay, which is why this is a change to where `barrier()` is called and not to any resource declaration.

API shapes were debated: a scoped “group” object whose passes share one barrier, and a mode flag on the encoder. The group form was rejected as too much API surface for the benefit, and per-call enable/disable was rejected because the scope of the switch is ambiguous at the call site — a concern the reporter later confirmed by forgetting to re-enable it. The resolution [11] is a flag fixed at encoder creation, which makes the mode a property of a whole encoder and lets an application put its serial work in one encoder and its independent work in another.

This paper’s **B-hazard** configuration is exactly that mechanism applied to a controlled workload, so the study can ask how far the field report generalizes. Section 6.2 shows that it does not generalize unconditionally, and Section 7 proposes the boundary.

5 Experimental Design

5.1 Configurations

Table 1 separates the controlled BLADE configurations from the realistic tracked baseline. **B-explicit-all** is a same-request insertion control: the two source paths issue the same timed global barrier calls by construction. The retained captures do not include this policy, so we do not present record equality as a capture-backed observation. **W-wgpu** uses the same shaders, resources, pass graph, and output checks, but remains an end-to-end comparison because the APIs and runtime guarantees differ.

WGpu is reported as an end-to-end tracked baseline because its validation, lifetime, binding, and command models differ from BLADE. Direct **wgpu-hal** measurements are optional diagnostics for separating frontend work; they are not required for the main comparison and

ID	Tracking	Placement	Scope (src/dst)
B-auto	none	every pass	broad/broad
B-hazard	none	app-declared	broad/broad
B-explicit-all	none	every pass	broad/broad
B-auto-scoped	none	every pass	kind/kind
B-hazard-scoped	none	app-declared	kind/broad
B-exp-all-scoped	none	every pass	kind/broad
W-wgpu	tracked	derived	per resource

Table 1: Evaluation configurations. Every **B-*** row uses a global memory barrier that names no resource and keeps images in **GENERAL**; automatic placement can derive both sides of a pass-kind scope, while an explicit barrier can derive only its source because the consumer is not yet known.

are not presented as another tracking implementation.

5.2 Workloads

The headless **sync-bench** artifact provides six workloads that vary dependency structure across compute, render, and alternating pass streams.

- **compute-independent**: one shared read-only input buffer, a distinct output buffer per pass. No inter-pass hazard.
- **compute-chain**: two ping-pong storage buffers. Every pass depends on its predecessor.
- **graphics-independent**: a distinct RGBA8 colour target per pass, cleared and stored. No inter-pass hazard.

- **graphics-chain**: one colour target loaded and stored with additive blending. Every pass depends on its predecessor.
- **mixed-independent** and **mixed-chain**: compute and render passes alternating, with the same resource patterns as the corresponding single-kind workloads applied within each family. Every pass boundary therefore joins two different pass kinds, which is where the scope axis has the most to give. In **mixed-chain** each pass depends on the pass of the same kind two positions earlier. From the third pass onward each incoming pass therefore has a real dependency; the collector’s hazard policy places a barrier before every pass except the first, so it differs from automatic placement by only the initial barrier, as the single-kind chains do. The barrier before the second pass is conservative—those first compute and render passes are independent—so “hazard” is a workload-level label for this mixed case rather than an edge-minimal schedule.

The two mixed families exist only in BLADE’s benchmark; the matched WGPU program has no equivalent, and they are used for the within-BLADE comparison rather than the end-to-end one.

Unless stated otherwise, each command buffer contains 16 passes; compute buffers hold 2^{20} `u32` elements, render targets are 1024×1024 , and each invocation performs 8 mixing rounds. Every configuration is warmed for 1000 iterations—chosen after the pilot’s ten warm-ups left severe within-block drift—and measured for 30, ten times, in an order randomized jointly across implementations. Residual drift is reported in Section 5.5; the warm-up count is not assumed to prove a fixed clock.

The matched WGPU program uses the same shader computations, resource sizes, pass graph, and iteration schedule. The interface declarations necessarily differ: BLADE supplies the 16-byte per-pass parameters as an inline uniform block, while WGPU uses its native **IMMEDIATES** feature (Vulkan push constants), so that the WGPU side does not gain an extra tracked uniform buffer. A checker in the artifact verifies that the shader bodies are textually identical after normalizing those interface declarations and binding decorations. After the final iteration of each process, the collector aborts if the two implementations produce different output hashes. All 1680 runs of the fixed matrix and all 8400 runs of the pass-count sweeps agree on the output hash within each collection/workload/pass-count group. Across the 154 such groups from all devices, every configuration present in a group agrees, with 0 exceptions.

Validation is layered so that a silent failure has somewhere to be caught. Each Vulkan machine’s round begins with a retained synchronization-validation matrix, run with the Khronos checks forced on and its logs stored beside the timing collection. After every timed process, a slice of every independent output is hashed with standard

FNV-1a and the collector aborts on any disagreement between policies or implementations. The graphics chain reserves its readback row for exact two-bit increments that cannot saturate across the 64-pass sweep — a missing late pass changes the digest instead of hiding under clamped bytes — while the other 1023 rows carry the full-range measured workload, and both implementations’ shaders pass SPIR-V validation.

Identical shader bodies do not guarantee identical generated code. BLADE hands naga the default bounds-check policies, which are unchecked throughout, and disables integer-division checks; WGPU’s `create_shader_module` keeps its own defaults, which clamp indices, guard divisions, and apply `force_loop_bounding` — a simulated 64-bit counter loaded, compared, decremented and stored on every iteration of a loop. The benchmark therefore uses WGPU’s unsafe trusted-shader entry point to select the unchecked policy that BLADE uses. This removes a shader-code confound, but it also means `W-wgpu` is a native tracked-WGPU comparison with one WebGPU safety feature class deliberately disabled, not a browser-WebGPU baseline. A separate `-shader-checks` diagnostic exists, but its earlier numerical results are not reported because that diagnostic collection is not present in the archival data.

These six are the whole workload set, deliberately. They contrast independent resource sets with chain dependency patterns across the pass kinds a synchronization policy can distinguish, and that is what the study is about. Application frames would add heterogeneity of a kind these do not have, and Section 8 states what that costs the conclusions; they would also add scene, content, and driver-heuristic variation that this design exists to exclude. We report a bounded claim about a controlled workload rather than an unbounded one about renderers.

They are also deliberately barrier-dense stress tests, and the two things they stress scale differently in a real application. A pass here performs only microseconds of work, so every boundary cost lands on the smallest denominator the design allows — sixteen boundaries against roughly a millisecond of total work. Fixed per-boundary costs — host-side barrier recording, the fraction-of-a-microsecond device-side scope constant — amortize in a production frame whose passes run hundreds of microseconds to milliseconds each: those results should be read as microseconds per boundary and divided by real pass durations, which shrinks them by one to two orders of magnitude. Overlap effects are structural instead: what a redundant barrier forfeits is proportional to how much genuinely independent work sits on both sides of it, which sixteen equal independent passes maximize and few real frames approach. On both axes a real application suffers less than these numbers; the field report is what the exception looks like when it occurs.

5.3 Platforms

Table 2 lists the machines. The matrix covers the minimum viable study of experiments.md — one recent AMD and one recent NVIDIA device-driver pair on Linux — and adds an Intel Vulkan implementation and an Apple/Metal case study. Results are reported per device-driver pair and never pooled by vendor.

One class is absent by construction: mobile GPUs. The matrix contains no Android-class tiler — no Adreno, Mali, or PowerVR — and while the AMD iGPUs and the Apple M3 are integrated parts, only the M3 is a tile-based deferred renderer, and it is measured through Metal’s tracker rather than through explicit barriers. On a Vulkan tiler, a barrier that forces a render-pass break costs a tile flush and reload — a mechanism none of the measured architectures has — so nothing in this paper should be extrapolated to that class; it is the most consequential gap in the platform set.

5.4 Metrics and statistics

We measure command-buffer start, pass recording, queue submission, and host wait separately, and report *host cost* as recording plus submission, because WGPU defers command translation and tracker resolution to `finish()` and `submit()` while BLADE performs it during recording. Charging only recording would misattribute the difference. The separately reported `start_ns` field is not included: it resets and reuses a BLADE backend encoder but creates a fresh WGPU frontend encoder, so it is a different lifecycle comparison rather than part of the record-and-submit estimand used here.

Device time is the GPU span of one command buffer. On Vulkan, BLADE writes a `TOP_OF_PIPE` timestamp before each pass and one after the final barrier; consecutive differences telescope, so their sum is exactly the span from the first pass to the end of the command buffer, and it remains well-defined when passes overlap. The WGPU program uses one beginning-of-first-pass and one end-of-last-pass timestamp. Thus BLADE emits 17 timestamp writes for a 16-pass run and includes its final barrier, whereas WGPU emits two writes and ends its window with the last pass. Both differences give BLADE more instrumented work, but they may also perturb scheduling; the instrumentation and windows are not identical. Only within-BLADE device comparisons are single-factor measurements.

The experimental unit for one configuration is a process launch, not one of the 30 samples inside it. One outer matrix repetition launches every configuration once in randomized order; for each of the ten outer repetitions we pair the contender launch with the `B-auto` launch from the same randomized block and compute their percent difference. The reported point is the median of those ten block effects; quoted absolute timings are likewise the median of the ten process medians. The 95% hierarchical-bootstrap percentile interval resamples outer blocks first

and observations within each selected process second, using a deterministic seed [5, 10]. Quartiles describe the pooled observations, but no interval treats all 300 as independent. With ten process launches per configuration, and possible serial correlation among observations inside a launch, these intervals describe uncertainty across the observed sessions rather than a sampled population — where launches disagree with one another, the interval says so by widening. We do not report multiplicity-adjusted hypothesis tests; the device-by-workload cells and their control floors are exploratory effect estimates. Validation layers are enabled for correctness runs and disabled for performance runs, as recommended by vendor guidance [19].

We did not preregister a practical-equivalence region, and in the end we do not use one. An earlier version of this analysis derived a single $\pm 3\%$ band from the worst `B-explicit-all` control deviation; with all 30 cells collected, control floors vary by cell and reach 106.7%, so no single band describes them. Every directional result is therefore compared against the control floor of the cell and pass count it comes from. Figures 3 and 4 draw the fixed-matrix floors beside the effects; Section 6.2 states the corresponding sweep floors. A band wide enough to cover the worst cell would discard the best ones, and a band fitted to the best would license the worst. This post-hoc floor is a conservative stability diagnostic, not a confidence bound or a preregistered equivalence test. Curve shapes and post-hoc state-conditioned summaries are explicitly labelled descriptive and are not counted as directional results.

5.5 Deviations from the protocol

These deviations affect how the results may be read, and are stated here rather than in a footnote.

Metal timestamps. BLADE’s Metal backend reports per-pass durations from counter sample buffers rather than a span, so summing them double-counts overlapping passes; the matched WGPU program returned zero for three of the four workloads. Metal device time is therefore not reported at all, and Section 6.8 uses host wall time. The machine also had macOS Low Power Mode enabled, which is recorded in the collection but not controlled.

Clocks that move during a measurement. Figures 3 and 4 carry a control floor for every cell: the disagreement between two configurations that issue the same global barrier calls by construction. Of the 30 cells, 10 have floors above 2%; the maximum is 106.7%, and the largest at or below two is 0.8%. A directional effect is treated as a result only when its entire interval lies beyond the same-side floor; otherwise it is not credited.

The pilot collections that preceded these showed severe monotone drift in their time-ordered samples. Those pilot rows are not part of the retained artifact, so they motivate the protocol but do not support a result. Each block — one workload, one policy, one repetition — is a sep-

Machine	Device	Backend	Driver	API	Unified	Platform	Revision
S1	RTX 5070	vulkan	NVIDIA 595.71.05	1.4.329	yes	Linux 7.0.0-28	87ed067
S2	RX 7900 XT	vulkan	radv 26.0.3-1ubuntu1	1.4.335	yes	Linux 7.0.0-15	87ed067
S2	Raphael iGPU	vulkan	radv 26.0.3-1ubuntu1	1.4.335	yes	Linux 7.0.0-15	87ed067
S3	Radeon 780M	vulkan	radv 25.2.8-0ubuntu0.24.04.1	1.4.318	yes	Linux 6.14.0-37	87ed067
S4	Intel Xe (RPL-U)	vulkan	anv 25.2.8-0ubuntu0.25.10.2	1.4.318	no	Linux 6.17.0-22	87ed067
S5	Apple M3	metal	Metal —	—	—	macOS 15.7.3	87ed067

The Vulkan rows were all collected on one day from one benchmark build: where their revisions differ, they differ only by commits under `paper/`, which the benchmark does not compile, and every row used the same WGPU revision. The Metal harness has only its backend’s automatic policy; the scope variants in this study alter Vulkan barriers and therefore do not apply to that row.

Table 2: Machines in the fixed 16-pass matrix. Every row is one collection under `paper/data/raw/`; results are never pooled across rows. “Unified” reports whether the driver advertises `VK_KHR_unified_image_layouts`, which Blade enables when present.

arate process launch, so the device idles between blocks and ramps its clock again inside each one. Ten warm-ups were visibly insufficient on the pilot RX 7900 XT even with `power_dpm_force_performance_level` set to `high`; increasing that count is a protocol parameter, not a benchmark change.

The retained collections use 1000 warm-ups, and comparing the median of each block’s first third with its last third checks that it worked: at the median block, drift is +0.1% on the RTX 5070, +0.0% on the RX 7900 XT, −0.0% on the Raphael iGPU, −0.0% on the Radeon 780M, and +0.0% on the Intel part. The worst single blocks still move — +8.6% on the RTX 5070 and −9.9% on the Intel part — which is the residue that the post-hoc control diagnostic is intended to flag.

The process repetitions expose what pooling hides. At three blocks the Intel part’s controls swung across signs and disqualified its cells; at ten its floors settle to 2.4%–5.0% of run-to-run variation and its compute-placement cells resolve (Section 6.2). The instability that remains in this round is a different machine and a different kind.

One host’s render cells vary launch to launch. On the Radeon 780M’s `graphics-independent` workload, paired block effects disperse far more between process launches than samples do within one launch, which is visible directly in the block marks of Figure 2. The host ran unrelated services during its multi-hour sweep, and that kind of interference is the mundane explanation we can neither exclude nor need to: the identical-request controls price the dispersion per pass count, the 16-pass control floor reaches 106.7%, and no conclusion is drawn from a cell whose interval fails its own floor. The directional support comes from the sweep counts whose full intervals clear their own controls, reported in Section 6.2.

Collections are one day, one benchmark build. Every machine, including the Apple M3, was measured on 2026-07-28 from the same corrected revisions of both repositories; where Table 2 lists a different BLADE revision for the study host’s sweeps, it differs only by commits under `paper/`, which the benchmark does not compile. The Metal collection has no scoped configurations

because those variants alter Vulkan barrier masks, not Metal’s framework-managed resource hazards.

6 Results

6.1 Control validity

The `B-exp-all` column of Table 3 is the instrumentation control: manual barriers before every pass reproduce the same global barrier calls by construction. A device-time difference is therefore measurement instability under an unchanged timed request. Its interval defines the paper’s conservative, post-hoc stability threshold for that cell, and it is drawn as a pair of ticks in Figures 3 and 4.

Two choices about that column matter. It is read per cell, because noise varies by workload even on one device. It is also the far end of the control’s hierarchical interval rather than its point estimate: the point can be near zero while different process repetitions occupy different regimes. Of the 30 cells, 10 have floors above 2%. That does not disqualify a whole device; it means the entire interval of a claimed effect must lie beyond the same-side floor of its own cell. The overlap-depth sweep applies the same construction separately at each pass count.

6.2 Barrier placement and dependency structure

Table 3 answers RQ2 and RQ4 together. The pattern is not “coarse barriers are slow”; it is conditional in a way that is stable across devices for the dependent workloads and unstable for the independent ones.

The chain comparison is an expected null. For the chain workloads, `B-hazard` places a barrier before every pass except the first, so the emitted stream differs from `B-auto` only by the initial barrier. Because both timing windows begin after that initial barrier, their timed barrier calls are the same by construction. Across the five Vulkan devices, 0 of the fifteen chain cells nevertheless resolve a difference above their own stability threshold. This is a useful check on the measurement, not

Device	Workload	B-auto	B-hazard %	B-exp-all %	W-wgpu %
RTX 5070	c-ind	195.4	-29.3 [-29.4, -29.2]	+0.0 [-0.1, +0.1]	+0.9 [+0.2, +1.2]
	c-chain	195.4	-0.1 [-0.1, +0.0]	-0.1 [-0.2, +0.0]	-0.4 [-0.5, -0.3]
	g-ind	152.1	-32.4 [-32.9, -31.8]	-0.1 [-0.2, +0.1]	-34.5 [-35.3, -33.6]
	g-chain	150.4	-0.1 [-0.7, -0.1]	-0.1 [-0.2, +0.1]	-17.5 [-17.6, -17.4]
	m-ind	172.1	-17.5 [-17.6, -17.4]	+0.0 [-0.2, +0.2]	—
	m-chain	171.8	-0.0 [-0.1, +0.1]	+0.0 [-0.1, +0.1]	—
RX 7900 XT	c-ind	120.6	-32.3 [-32.6, -32.1]	+0.3 [-0.2, +0.5]	-29.9 [-30.2, -29.8]
	c-chain	126.2	-0.5 [-2.3, +0.6]	-1.1 [-2.3, +0.8]	+0.0 [-0.1, +1.8]
	g-ind	178.0	-7.3 [-7.5, -7.2]	-0.1 [-0.2, -0.0]	+12.0 [+11.9, +12.3]
	g-chain	150.8	+0.1 [-0.0, +0.2]	-0.1 [-0.1, +0.0]	+18.5 [+18.3, +18.6]
	m-ind	155.9	-17.7 [-17.8, -17.7]	-0.0 [-0.1, +0.1]	—
	m-chain	141.8	-0.1 [-0.1, +0.1]	-0.1 [-0.2, +0.1]	—
Raphael iGPU	c-ind	3854.7	-0.9 [-0.9, -0.8]	-0.0 [-0.0, +0.1]	+6.1 [+6.1, +6.2]
	c-chain	3833.6	+0.0 [-0.0, +0.1]	+0.0 [-0.0, +0.0]	+7.4 [+7.4, +7.5]
	g-ind	5724.6	-1.1 [-1.1, -1.1]	-0.0 [-0.1, +0.0]	-13.4 [-13.4, -13.4]
	g-chain	4829.8	-0.0 [-0.0, +0.0]	-0.0 [-0.0, +0.0]	+2.4 [+2.4, +2.4]
	m-ind	4803.9	-1.8 [-1.9, -1.8]	-0.0 [-0.0, +0.0]	—
	m-chain	4375.6	+0.0 [-0.0, +0.0]	-0.0 [-0.0, +0.0]	—
Radeon 780M	c-ind	3597.8	+3.6 [+3.0, +4.0]	-0.1 [-0.3, +0.7]	+2.4 [+2.1, +2.7]
	c-chain	3600.3	-0.0 [-0.4, +1.5]	-0.3 [-0.6, +0.2]	+5.2 [+4.9, +5.4]
	g-ind	2322.8	+42.0 [+22.5, +184.6]	+51.9 [-14.5, +106.7]	-12.2 [-28.4, -10.7]
	g-chain	1465.2	-0.1 [-1.4, +0.5]	+0.1 [-0.8, +0.6]	-0.2 [-2.2, +7.6]
	m-ind	2944.0	+25.7 [+24.8, +57.4]	-0.2 [-23.9, +0.1]	—
	m-chain	2890.4	+0.0 [-0.1, +0.4]	+0.1 [-0.4, +51.6]	—
Intel Xe (RPL-U)	c-ind	4017.0	-6.5 [-7.5, -5.3]	+0.1 [-0.6, +2.4]	+11.7 [+8.1, +12.3]
	c-chain	3594.0	+0.0 [-0.0, +0.8]	-0.0 [-4.9, +0.0]	+12.9 [+10.2, +13.7]
	g-ind	4256.7	-0.4 [-0.5, +0.1]	+0.1 [-0.0, +3.5]	+7.9 [+3.9, +8.7]
	g-chain	3324.9	-0.0 [-3.4, +0.0]	-0.0 [-4.4, +0.0]	+4.9 [-0.3, +4.9]
	m-ind	4098.5	-8.2 [-11.3, -7.4]	+0.1 [-3.7, +4.0]	—
	m-chain	3829.7	+2.1 [-2.5, +5.1]	+2.0 [-4.6, +5.0]	—

Every cell compares the two implementations on the same physical device; the collector aborts otherwise (Section 5.5).

Table 3: GPU span of one 16-pass command buffer on Vulkan: **B-auto** median of process medians in microseconds, then the median paired process-level percent differences with 95% hierarchical-bootstrap intervals. **B-explicit-all** is the instrumentation control: it issues the same timed global barrier calls as **B-auto** by construction. The far endpoint of its interval is used as a conservative, post-hoc stability threshold (Section 5.5). **W-wgpu** is an end-to-end comparison with two timestamps ending at the final pass, whereas BLADE writes one per pass and includes its final barrier in the span.

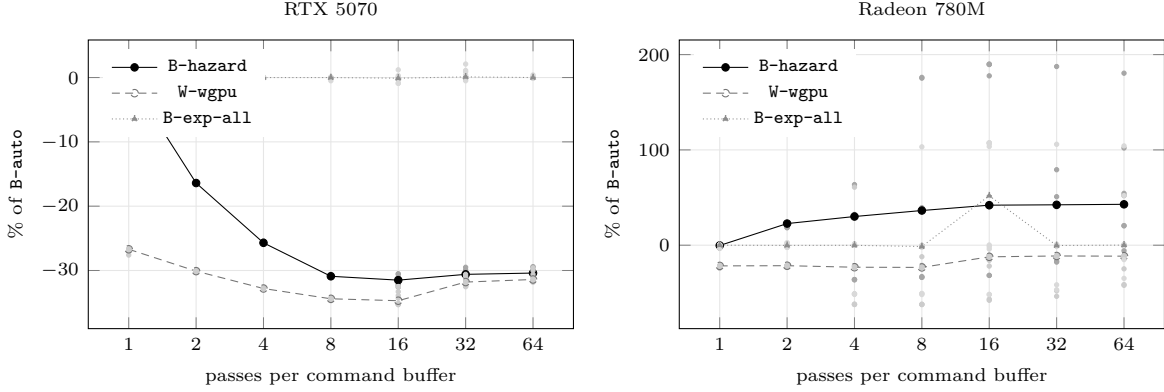


Figure 2: GPU-span effect of removing the redundant barriers (B-hazard) and of the tracked implementation (W-wgpu) against B-auto on **graphics-independent**, as the number of overlapping passes grows. B-exp-all is the identical-request control. Each small mark is one paired process launch; the line joins the medians of those block effects. The scattered marks on the Radeon 780M panel are that host’s launch-to-launch dispersion (Section 5.5); directional results use the count-specific control intervals stated in the text, while the joined medians describe the sweep’s shape.

evidence that a required barrier is free. The independent workloads are the placement experiment: there B-hazard removes the fifteen timed inter-pass barriers and effects reach 32.4%.

When the barrier is redundant, the cost depends on the device and on the kind of work. For independent *compute* passes, removing the redundant barriers is worth 29.3% [29.2, 29.4] on the RTX 5070 and 32.3% [32.1, 32.6] on the RX 7900 XT. The reductions are consistent with recovered overlap and/or avoided pipeline drains and cache work; the timestamp span alone does not distinguish those mechanisms. Both figures moved between the ramping pilot and this warmed collection — the AMD one grew, and its control floor fell from a quarter of the effect to 0.5%.

The Intel part, unreadable at three blocks, resolves at ten: -6.5% $[-7.5, -5.3]$ on **compute-independent** against a floor of 2.4%, with -8.2% $[-11.3, -7.4]$ on **mixed-independent** against 4.0% agreeing — a third vendor for the direction of the compute result, at well under the NVIDIA magnitude.

For independent *render* passes the same change is worth 32.4% [31.8, 32.9] on the RTX 5070 but only 7.3% [7.2, 7.5] on the RX 7900 XT — resolved against a floor of 0.2% in the observed sessions, and a quarter of what the same machine recovers on compute — and -1.1% $[-1.1, -1.1]$ on the Raphael iGPU, resolved and still nearly nothing. (The pilot could not separate the RX 7900 XT’s render cell from zero; the warmed collection yields a small directional estimate.) The sharpest split in the study is therefore not vendor against vendor but kind of work on the same silicon: the RX 7900 XT frees four times more span between compute passes than between render passes, while the RTX 5070 frees the same third either way. The headline “placement dominates” is carried by both discrete parts for compute, and at full

size by NVIDIA alone for graphics.

Removing barriers can make things worse, and the sweep shows where the claim resolves. On the Radeon 780M, B-hazard costs at 3 pass counts under the same criterion used for the fixed matrix. The effect is absent at one pass (-0.3%). At two passes its full interval clears a 2.2% control floor. At 32 passes the effect is $+42.4\%$ $[+40.5, +59.2]$ against a 12.3% floor, and at 64 it is $+42.9\%$ $[+30.2, +100.6]$ against 18.6%.

The medians at four, eight, and sixteen passes suggest growth and saturation near sixteen, but their intervals do not clear their count-specific floors; that curve shape is descriptive rather than a fourth result. In particular, the 16-pass point ($+42.0\%$) is not promoted: that count’s launch-to-launch dispersion (Section 5.5) puts its floor past its interval.

That descriptive growth is what a shared-resource account predicts, but the tracked comparison makes the simplest version inadequate. WGPU overlaps the same passes onto usage-matched targets (both request attachment and copy usage only, so both are DCC-eligible by the predicates of Section 2.2), writes the same bytes, and has a faster point estimate than serialized B-auto across the sweep: -21.5% at two passes, -11.5% at sixty-four. This descriptive contrast argues against overlap depth or target bytes alone as the explanation; it does not identify the resource involved. What still differs between the implementations here is narrow: the layout the targets sit in (GENERAL against COLOR_ATTACHMENT_OPTIMAL) and the endpoint barrier form — on a driver that advertises `VK_KHR_unified_image_layouts` and therefore promises that the layout half should not matter — along with parameter delivery, timestamp instrumentation, and backend-specific per-pass work. Under hazard-only placement there is no interior barrier left to blame. The controlled layout variant — the same BLADE stream with

attachment-optimal layouts — is the experiment that would close this, and the study does not have it. We report the shape and the eliminations rather than the tidy explanation the numbers do not support.

Long-running passes leave little measured headroom. On the Raphael iGPU the largest highlighted within-BLADE change is 1.8%, and every floor is at or below 0.1%, so the small effects are resolved where their intervals and thresholds permit. This is consistent with one pass saturating the small GPU and leaving no capacity for a second, although utilization was not measured directly. What does move on this device is the tracked implementation — by up to 13.4%, in both directions across workloads — which the saturation account alone would not explain, and is taken up in Section 6.7.

This suggests a boundary the field report does not reveal on its own. Overlap can pay when adjacent passes have complementary bottlenecks and the device has idle capacity — ray tracing next to rasterization, in the reported case. It need not pay when each pass already saturates the same resource. The Radeon result also shows that removing a barrier can lose, although this experiment does not establish memory pressure as the cause.

6.3 Barrier scope at fixed placement

Figure 4 isolates the axis introduced in Section 3.1. The scoped policies emit the same number of barriers at the same boundaries as their unscoped twins; only the declared stages and accesses change. The two chain workloads make the separation sharp, because there every timed inter-pass barrier is retained, so any difference between the automatic scoped and unscoped policies must come from scope.

The result splits the prediction that motivated the configuration down the middle, and both halves are informative. It also corrects this paper’s own pilot, whose floors on the decisive AMD cells were wider than the effect and which we initially summarized as “nothing measurable on AMD”. That summary was a statement about the pilot’s noise, not about the device; the warmed collection resolves what it could not.

Where it pays. On the RTX 5070 narrowing the scope is worth 5.0% [4.8, 5.0] on **graphics-chain** and 1.2% [0.3, 3.4] on **mixed-chain**, against control floors of 0.2% and 0.1% in those cells. It is not confined to chains on that device: the same change is worth 4.9% [4.7, 5.0] on **graphics-independent**, floor 0.2%, where it is available *in addition* to the much larger placement effect. And it now pays on AMD — on the discrete part, at compute-to-compute boundaries only: 6.7% [4.5, 6.9] on **compute-chain** against a floor of 2.3%, confirmed from a far cleaner cell by 4.7% [4.5, 5.2] on **compute-independent**, floor 0.5%. At this 16-pass point **compute-chain** falls from 126.2 μ s to 117.7 μ s, while **compute-independent** falls from 120.6 μ s to 115.0 μ s. We did not sweep pass count on this device, so their similar

absolute savings do not establish a fixed per-boundary law.

Threshold-edge iGPU observations. Raphael’s two compute cells have one-signed intervals: $-0.2\% [-0.3, -0.1]$ and $-0.2\% [-0.2, -0.1]$, against floors of 0.1% and 0.0%. The **compute-independent** interval still touches its stability band; the **compute-chain** interval clears its band only at the rounding edge. Either way these are fractions of a percent on a saturated device, not the practical claim made for the discrete part.

Where no direction resolves. Every RX 7900 XT boundary that involves a render pass on either side fails the paper’s exploratory directional criterion: $-0.1\% [-0.2, +0.0]$ on **graphics-independent** at a floor of 0.2%, with no resolved directional effect on **graphics-chain** or either mixed workload. This is not an equivalence result. On the Radeon 780M the two compute cells read $+0.2\% [-0.1, +0.8]$ and $-0.1\% [-0.5, +0.3]$ against floors of 0.7% and 0.6%, and neither clears its cell’s floor. The Intel part is not credited in either direction. Its **compute-chain** column reads $-0.1\% [-1.6, +5.0]$ against a floor of 4.9% (Section 5.5). Its **graphics-independent** estimate is $-0.3\% [-0.5, +2.9]$, but that wide interval overlaps the $\pm 3.5\%$ stability band. The pilot’s Intel **graphics-chain** effect, once quoted here as the Intel result, did not reproduce ($-0.6\% [-0.6, +2.0]$).

The pattern points to the destination scope. The mixed workloads localize it. Every **mixed** boundary joins two pass kinds, so its scoped barrier narrows the *source* exactly as a compute-to-compute barrier does — and no mixed AMD cell resolves a directional effect. What a mixed boundary never has is a compute-only *destination*. The crossed policies point the same way from the other side: **B-hazard-scoped**, whose explicitly placed barriers narrow their source but must keep a wide destination (Section 3.1), shows little direct scope effect: $-0.6\% [-1.8, +1.4]$ on the AMD compute chain, where automatic scope is worth $-6.7\% [-6.9, -4.5]$, and $+0.1\% [+0.0, +0.3]$ on the NVIDIA graphics chain, where automatic scope is worth $-5.0\% [-5.0, -4.8]$. Together these comparisons are consistent with the saving depending on the narrowed destination unavailable to an explicit barrier. They localize the pattern to that side of the request; they do not identify the driver operation responsible.

This is not the shape the prediction had. Section 2.2 derives from RADV that narrowing sheds the most at render-involving boundaries — a pixel drain, two attachment flushes, two metadata flushes — and it is right about the commands; that part is read from the source. But those boundaries show no resolved saving on any AMD part, while the compute-to-compute boundary, where the shed operations run against idle pixel hardware and clean caches and were priced accordingly, is where the discrete part actually pays. The drivers whose source we did not

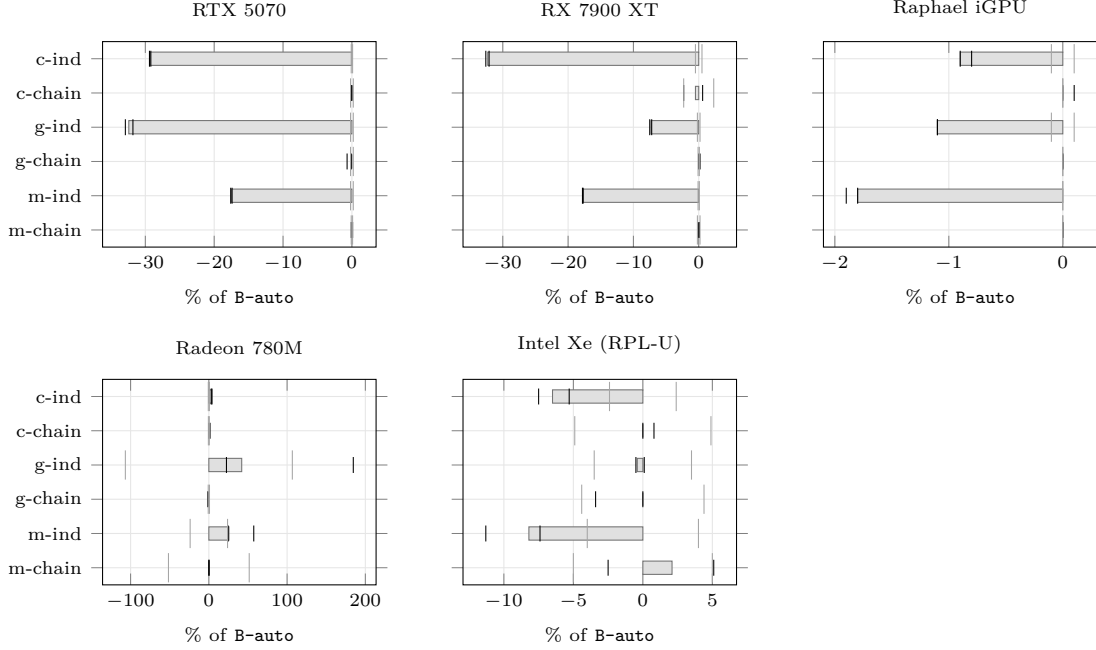


Figure 3: Barrier placement: percent difference in GPU span between **B-hazard** and **B-auto** at 16 passes, by device and workload. The bar is the median paired process effect, the two black ticks its 95% hierarchical-bootstrap interval, and the wider grey ticks plus and minus that cell’s post-hoc stability threshold. A directional effect is read only when its whole interval lies beyond the corresponding grey tick. Note the per-panel horizontal scales.

read split the same way for their own reasons: broad savings on NVIDIA, nothing creditable on Intel.

Going back to the source explains most of it (Section 2.2.1): on GFX10 and later with a coherent TCC, `radv_dst_access_flush` drops every destination-side L2 invalidation unless a render-backend-incoherent attachment is dirty. If the measured targets did not set that pipe-misaligned state, half of what the narrower barrier was designed to save was already being discarded by the driver without the application saying anything; the benchmark did not instrument that internal flag, so the account remains conditional. On the source side, a flush costs time only when there is matching work in flight or dirty state. A render boundary retains its attachment-flush requests under either scope, while a pure compute stream has no attachment work. This supplies a plausible source-level account of the unresolved render-boundary effects, not a counter measurement. What it does not account for is the compute-boundary saving itself: the operations our reading priced there are the free-when-idle ones, and the measurement instead follows the destination mask — a dependence the reading never enumerated, because everything it enumerated on the destination side was cache work the driver had already elided. We can name the side the cost lives on and not, from the text we read, the operation.

The general mistake is worth naming, because it is easy to repeat and we made it in the obvious way. Counting the commands a barrier expands into records an over-approximate request set; it is not a numerical upper

bound on elapsed cost. Here that request set is longest exactly where its timing effect is least resolved. We read the half of the driver that translates a request and not the half that decides what to do about it. Reading it was still worth doing — it produced the configuration, and it explains most of the result — but the explanation arrived after the measurement, and would not have been trusted before it.

Section 6.9 checks the relevant request with captures: the extracted barrier tables on all 4 machines are byte-identical. The spread — 5.0% on one vendor’s render chain, 6.7% on another’s compute chain, no resolved direction at any AMD render boundary — therefore is not explained by different masks or barrier counts in the extracted records. The captures do not establish that the complete command streams or driver-internal work are byte-identical.

Noise is a property of the cell, not the device. The floor ticks in Figures 3 and 4 make this legible. The Intel part ranges from a floor of 3.5% on one cell to 5.0% on another in the same session. The RX 7900 XT also varies by workload, from 0.2% on its cleanest cell to 2.3% on a less stable one. A per-device maximum would discard usable cells; a pooled sample bootstrap or the control’s point estimate would license unstable ones.

Manual placement forfeits the scope saving. The “both” column is therefore not the sum of the first two: combining hazard-only placement with the narrow scope resolves the placement effect but no additional scope saving, in every cell of every vendor where scope pays, for the

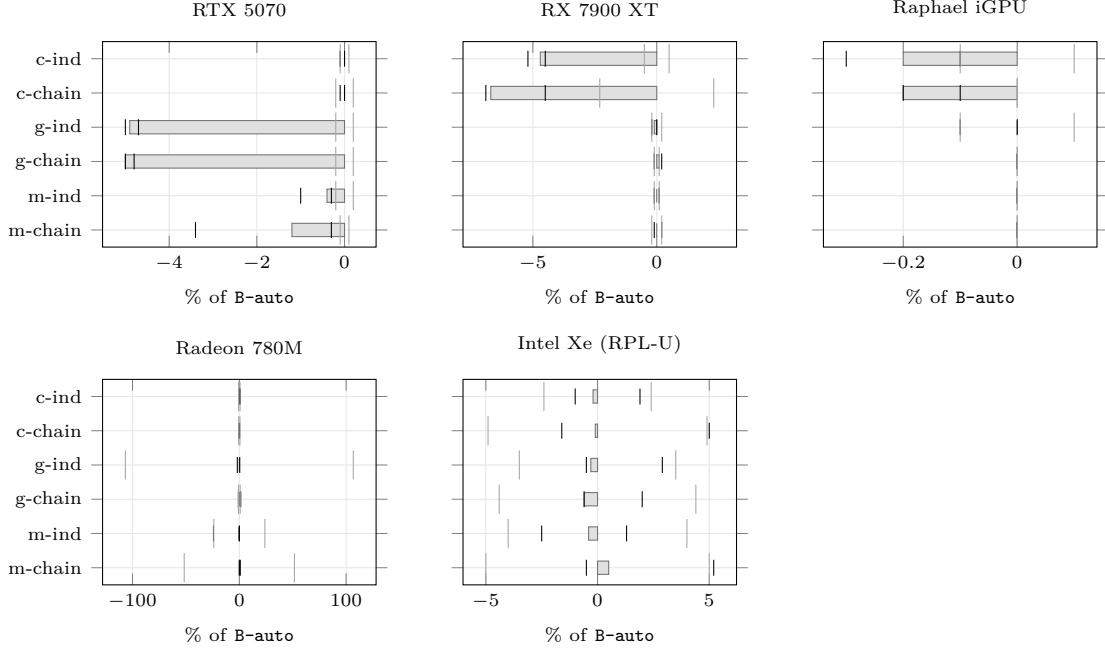


Figure 4: Barrier scope at fixed placement: percent difference in GPU span between **B-auto-scoped** and **B-auto**, drawn as in Figure 3. The chain workloads are the clean test, because every timed inter-pass barrier is retained there. The grey threshold comes from the global identical-request control; it is a cell-stability diagnostic, not a scoped control.

reason Section 3.1 builds in — an explicitly placed barrier narrows its source but not its destination, matching the destination-side pattern above. That is the opposite of the intuition that more control is strictly better. An application that leaves placement to the encoder gets a barrier the encoder can describe on both sides; one that takes placement into its own hands gets a barrier that only half describes itself. The scope saving is also the cheaper of the two gains to obtain — no API change and no resource tracking — so where it exists it should be taken first, and where placement is also taken the two should not be assumed to compose.

6.4 Scaling with pass count

Table 4 sweeps the pass count on the RTX 5070 and answers RQ1. GPU span has nearly constant increments across the whole range. Host medians are well behaved only over the 1–8-pass range used for the reported average increment; Figure 5 compares those endpoint averages directly. The 16-pass values are reported as measured but lie outside that host summary range.

On the host side, the endpoint averages imply an additional global-barrier cost of $0.74\mu\text{s}$ per pass for compute (1.10 against 0.36) and $0.58\mu\text{s}$ for graphics (1.48 against 0.90). The matched WGPU program averages 5.59 and $3.75\mu\text{s}$ per pass, which is $5.1\times$ and $2.5\times$ BLADE’s automatic policy and $15.5\times$ and $4.2\times$ BLADE with no inter-pass barrier. At sixteen passes the absolute gap is 54.5 against $230.2\mu\text{s}$.

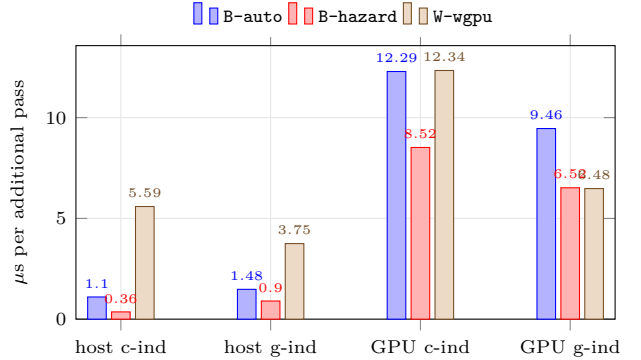


Figure 5: Average cost of one additional pass on the RTX 5070, from the pass-count sweeps. Host figures come from the timestamp-free collection and device figures from the GPU-timed one; each is the endpoint increment over the range stated in Table 4. The gap between **B-auto** and **B-hazard** is what one redundant barrier costs; the gap to **W-wgpu** is end-to-end.

Past eight passes the host cost leaves that linear regime: between the 8- and 16-pass medians it jumps several-fold on both implementations, and beyond sixteen it grows steeply but smoothly. Something in the submission path changes mode near that size, and its onset wandered between collections — the superseded three-block sweep crossed over between sixteen and thirty-two, erratically — so no single slope describes the full range. The av-

	Config	1	2	4	8	16	32	64	$\mu\text{s}/\text{pass}$
<i>compute-independent</i>									
host	B-auto	2.8	3.6	5.2	10.5	54.5	142.0	269.3	1.10
host	B-hazard	2.5	2.8	3.5	5.0	16.0	64.4	122.9	0.36
host	W-wgpu	6.3	9.0	14.2	45.4	230.2	383.6	672.6	5.59
GPU	B-auto	10.9	21.1	43.5	86.6	195.3	391.8	785.0	12.29
GPU	B-hazard	10.9	19.2	35.8	70.8	138.2	274.3	547.9	8.52
GPU	W-wgpu	11.0	22.6	47.0	96.3	196.5	394.5	788.3	12.34
<i>graphics-independent</i>									
host	B-auto	3.4	4.7	7.2	13.8	62.2	215.9	415.5	1.48
host	B-hazard	3.1	4.0	5.8	9.4	27.7	147.2	306.1	0.90
host	W-wgpu	7.3	10.8	17.7	33.6	115.0	195.6	356.0	3.75
GPU	B-auto	10.4	19.7	38.3	75.6	151.6	303.0	606.4	9.46
GPU	B-hazard	10.4	16.5	28.4	52.3	103.8	209.8	421.1	6.52
GPU	W-wgpu	7.6	13.8	25.7	49.6	99.3	206.3	416.1	6.48

GPU span has nearly constant increments across the whole range. Host cost is linear only to 8 passes: between 8 and 16 its medians jump several-fold into a second, steeper regime, so one slope fitted over all counts would describe neither. The 1–8 average characterizes the linear regime; the larger counts are reported as measured.

Table 4: Pass-count sweep on the RTX 5070, medians of process medians in microseconds. Host rows come from the timestamp-free collection; GPU rows come from the GPU-timed collection. The last column is the average increment per additional pass between the endpoint medians, taken over the whole range for the GPU rows and over 1–8 passes for the host rows.

erages above are therefore restricted to 1–8 passes; the 16-pass matrix is a directly measured point inside the second regime, not an extrapolation from the first.

On the device side, the corresponding endpoint-average cost of each additional redundant barrier is $3.77\mu\text{s}$ of GPU span for compute (12.29 against 8.52 per pass) and $2.94\mu\text{s}$ for graphics (9.46 against 6.52). Against passes that themselves cost 8.52 and $6.52\mu\text{s}$, that is more than a third of a pass lost per boundary. Within this repeated-pass sweep, the auto–hazard gap is descriptively visible at every measured count above one; at one pass there is no timed inter-pass barrier to remove. This is not a claim about other graph shapes or devices.

The timestamp-free sweep exists to check whether GPU timing perturbs the host measurement — BLADE records one timestamp per pass against two in total for WGPU, so any effect would also be unmatched between them. On this driver, at ten repetitions, there is none to find: the timestamped and timestamp-free host medians agree at every pass count for both implementations. A superseded three-block sweep had appeared to show a gap growing with the pass count; it did not survive the repetitions. The fixed matrix’s host columns are therefore read as host cost, with the instrumentation concern tested on this machine and untested on the others.

6.5 Host cost across devices

Table 5 repeats the host comparison on every machine. Over the 24 shared cells, the matched WGPU program uses between $1.3\times$ and $5.9\times$ the host time of BLADE’s automatic policy for the same sixteen passes, and the direction is the same in every one of them. These collections have GPU timestamps enabled, which charges BLADE one

`vkCmdWriteTimestamp` per pass against two in total for WGPU. That extra recorded work plausibly biases against BLADE, but the two timestamp paths differ enough that we do not treat the comparison as a formal lower bound. The ratios describe this instrumented end-to-end path, not an uninstrumented record-only cost; the timestamp-free RTX 5070 sweep supplies the clean magnitude check. The sign never changes in the fixed matrix.

The narrowest Vulkan ratios are the NVIDIA render cells, and the **B-hazard** column says why: that driver charges heavily for recording the barriers themselves, so removing them cuts BLADE’s own host cost by 54.5% on **graphics-independent**. Where BLADE’s denominator is mostly barrier recording — a cost the application controls — the ratio narrows; nowhere does it invert. The Metal rows are narrower still for the backend’s own reason: each BLADE pass there creates a fresh command encoder (Section 6.8), which inflates the same denominator.

The difference is end-to-end and must not be attributed to tracking alone: WGPU also validates, manages lifetimes, resolves bind groups, and builds an intermediate command representation. It is also not a barrier-count effect: on the independent workloads Section 6.9 counts 5 barriers from **B-auto** against 1 from WGPU, and WGPU still costs several times more host time.

6.6 Where the host time goes

Table 6 attributes each implementation’s CPU time to the crate or library that spent it. The workload was shaped to emphasize host work — tiny dispatches, small targets, many passes, no timestamp queries — but the profile samples the whole process, including a completion wait after every iteration. It therefore did not isolate the recording

Device	Workload	B-auto	B-hazard %	W-wgpu %
RTX 5070	c-ind	56.0	-61.5 [-63.2, -58.2]	+282.3 [+116.2, +339.4]
	c-chain	58.3	-11.3 [-22.1, -2.6]	+270.8 [+224.2, +322.2]
	g-ind	70.0	-54.5 [-58.5, -50.2]	+59.4 [+46.2, +73.6]
	g-chain	60.3	-11.3 [-36.8, -1.7]	+128.7 [+115.5, +144.9]
	m-ind	69.3	-46.7 [-51.2, -42.1]	—
	m-chain	67.1	-0.9 [-13.8, +10.2]	—
RX 7900 XT	c-ind	10.6	-14.8 [-16.2, -13.5]	+429.1 [+417.0, +431.9]
	c-chain	10.6	-0.5 [-1.0, +0.7]	+412.7 [+410.1, +420.5]
	g-ind	27.9	-3.0 [-4.8, -2.6]	+211.2 [+207.2, +214.3]
	g-chain	19.7	-0.6 [-1.6, +0.2]	+288.0 [+285.1, +290.4]
	m-ind	20.0	-6.6 [-8.0, -5.9]	—
	m-chain	15.9	-1.9 [-3.1, +0.0]	—
Raphael iGPU	c-ind	8.8	-14.9 [-18.9, -12.1]	+486.0 [+458.3, +499.3]
	c-chain	8.7	-3.5 [-9.0, +3.7]	+490.7 [+456.8, +502.9]
	g-ind	30.5	-4.4 [-5.6, -3.8]	+163.7 [+161.2, +168.0]
	g-chain	17.7	-0.6 [-2.6, +0.5]	+312.8 [+307.6, +318.6]
	m-ind	20.0	-6.2 [-7.5, -5.9]	—
	m-chain	13.7	+0.2 [-0.5, +1.3]	—
Radeon 780M	c-ind	68.6	-3.7 [-6.3, +2.2]	+246.2 [+239.6, +271.9]
	c-chain	67.3	-0.9 [-3.3, +1.8]	+250.9 [+243.3, +257.5]
	g-ind	112.8	+0.8 [-2.5, +43.4]	+162.7 [+44.0, +168.4]
	g-chain	29.0	+2.2 [-49.4, +125.8]	+374.9 [+58.9, +582.1]
	m-ind	101.5	+0.6 [-2.8, +22.5]	—
	m-chain	90.5	+1.8 [-1.0, +8.8]	—
Intel Xe (RPL-U)	c-ind	95.8	+0.2 [-1.6, +16.9]	+297.5 [+266.1, +308.2]
	c-chain	83.7	-4.4 [-8.9, +6.8]	+324.0 [+309.8, +336.7]
	g-ind	240.4	-0.4 [-4.8, +3.8]	+122.5 [+114.5, +129.5]
	g-chain	132.1	-0.9 [-4.4, +5.0]	+200.8 [+191.2, +209.6]
	m-ind	220.4	+3.5 [+0.9, +10.2]	—
	m-chain	185.4	+1.5 [-11.7, +4.3]	—
Apple M3	c-ind	68.3	—	+33.7 [+24.5, +57.9]
	c-chain	68.3	—	+33.0 [+24.8, +42.9]
	g-ind	85.2	—	+77.6 [+64.3, +98.2]
	g-chain	83.8	—	+79.3 [+71.0, +86.0]

Percent differences rather than ratios: a **W-wgpu** entry of +200% is three times **B-auto**.

Table 5: Host cost of one 16-pass command buffer: recording plus submission, median of process medians in microseconds, then percent differences from **B-auto** with 95% hierarchical-bootstrap intervals over paired process repetitions. These collections have GPU timestamps enabled. Blade records one `vkCmdWriteTimestamp` per pass against two in total for wgpu, so this is an instrumented end-to-end comparison, not a matched timestamp workload.

Component	S1				S2				S3				S4			
	c-ind		g-ind		c-ind		g-ind		c-ind		g-ind		c-ind		g-ind	
	B	W	B	W	B	W	B	W	B	W	B	W	B	W	B	W
kernel	45.5	36.1	41.1	30.2	87.9	34.6	10.1	24.7	33.5	52.3	15.0	35.2	39.4	7.8	27.7	6.6
driver	29.2	40.5	46.1	36.7	5.9	24.8	80.3	35.3	31.0	21.4	74.7	36.6	42.5	29.4	57.2	36.3
ash / loader	0.3	0.1	0.1	0.1	0.1	0.1	—	—	0.5	—	0.1	—	0.3	—	—	—
wgpu tracker	—	5.1	—	3.1	—	18.6	—	6.8	—	4.7	—	1.5	—	4.2	—	2.0
wgpu command	—	6.2	—	8.7	—	7.7	—	13.5	—	5.6	—	9.4	—	4.4	—	5.8
wgpu-hal	—	1.1	—	1.9	—	1.5	—	3.3	—	1.4	—	2.3	—	0.8	—	1.7
wgpu device/resource	—	1.2	—	1.0	—	2.0	—	1.3	—	0.8	—	1.1	—	1.3	—	1.1
wgpu validation	—	0.2	—	—	—	0.1	—	0.1	—	0.1	—	0.2	—	0.1	—	0.1
wgpu other	—	0.8	—	0.8	—	1.3	—	0.8	—	0.6	—	0.4	—	0.6	—	0.6
blade	20.7	0.2	1.9	0.2	5.1	0.3	2.2	0.4	25.9	0.1	1.8	0.1	3.3	0.2	1.9	0.1
allocator	0.7	3.5	0.8	3.8	0.1	1.3	0.6	0.8	1.0	2.1	0.5	1.6	1.8	2.6	1.1	2.7
libc / runtime	3.7	3.9	4.0	5.5	0.9	5.6	7.3	8.1	6.1	6.3	9.2	6.8	8.3	44.8	5.8	39.1
other	1.0	3.4	4.0	6.0	0.6	3.1	1.7	2.7	1.7	3.7	1.5	2.4	5.1	2.1	1.8	2.3

Shares within a process, not times between processes: **task-clock** charges a blocking fence wait to the process whenever the driver spins inside it, which inflates the *driver* row differently for the two implementations. Self time is attributed to the symbol a sample landed in, so inlined tracker work can be charged to its caller; the tracker-labelled share can therefore undercount, but is not a formal lower bound. Columns sum to slightly more or less than 100 because **perf** rounds each symbol's share independently. The *driver* row is S1 on `libnvidia-eglcore.so.595.71.05`, S2 on `libvulkan_radeon.so`, S3 on `libvulkan_radeon.so`, S4 on `libvulkan_intel.so`. The columns are labelled by machine and driver rather than by device; only shares within one process are read from them. Because waits dominate these whole-process profiles, the component shares cannot apportion the record-and-submit gap.

Table 6: Share of process CPU time by component, from flat **perf** profiles. B is BLADE and W the corresponding WGPU program. The workload uses tiny dispatches, small targets, many passes, and no timestamp queries, but the profile covers the whole process, including completion waits.

interval as intended.

That limitation changes what the table can support. Symbols assigned directly to WGPU's resource tracker account for 1–19% of its process samples, and all WGPU buckets together account for 11–31%. The tracker-labelled figure likely undercounts inlined tracker work, which is charged to its caller, but it is also a share of a denominator dominated by waits and driver activity. It cannot establish either that tracking explains the record-and-submit gap or that it is too small to explain it.

What dominates BLADE's whole process is the kernel and the driver: 65–94% in every column. The same two rows take 37–77% of WGPU's — its largest share everywhere except the Intel machine, where a single symbol out-bills them: the *libc / runtime* row reaches 44.8% of the WGPU process, almost all of it `__memset_avx2_unaligned_erms` — something in that stack clears a lot of memory on ANV, and a flat profile cannot name the caller — against 8.3% for all of *libc* in BLADE's column on the same machine. The totals are also not comparable between implementations — **task-clock** charges a blocking fence wait to the process whenever the driver spins inside it, and BLADE and WGPU wait differently — which is why the table reports shares within a process rather than times between processes.

Repeating one profile moves shares by around a percentage point, so small row differences are not meaningful. The profiling invocations were not adapter-pinned, so on a multi-GPU host the two implementations need not

have profiled the same device; only within-process shares are read from the table for that reason. A causal decomposition of the host gap needs a follow-up that gates sampling to record plus submit (or batches many recordings before one wait) on the same pinned device. These flat whole-process profiles are retained as a diagnostic, not presented as that decomposition.

6.7 End-to-end Blade versus wgpu

The W-wgpu column of Table 3 is more interesting than a single ratio suggests, because its sign changes — between devices, and on one device between kinds of work.

On the RTX 5070 the matched WGPU path follows the same direction as application-declared placement. On **graphics-independent** it is 34.5% [33.6, 35.3] faster than **B-auto**: it knows the sixteen render targets are distinct, emits no dependency between the passes, and lands within a couple of points of BLADE's own declared placement (32.4%) — a residue of the same order as the destination-scope saving that an explicitly placed barrier forfeits (Section 6.3). That numerical proximity is descriptive: the barrier form, parameter interface, command machinery, and timestamp instrumentation still differ. The host column of Table 5 separates the paths much more sharply.

On **graphics-chain** the WGPU path is 17.5% [17.4, 17.6] faster than **B-auto** on a workload where every boundary is a real hazard, so the within-BLADE placement variants have identical timed barriers

and scope recovers only 5.0%. The rest of the gap — well over twice what scope recovers — remains bundled among the resource-scoped form of the barrier, the specialized image layout, differing timestamp windows, and per-pass work outside synchronization. This driver advertises `VK_KHR_unified_image_layouts` and BLADE enables it, which is a reason not to expect a `GENERAL` penalty in supported uses; we observe the gap and cannot apportion it. Neither compute cell shows a resolved WGPU advantage, so the large end-to-end difference observed here is specific to the render path.

On the discrete AMD part the comparison runs the other way. The `compute-independent` WGPU result is close to declared placement (29.9% faster than `B-auto`, beside 32.3% for `B-hazard`), while its render path is 12.0% slower on `graphics-independent` and 18.5% slower on `graphics-chain` — slower, that is, than a BLADE baseline whose global barriers prevent inter-pass overlap. The current capture extract identifies barrier counts, kinds, masks, and layouts but does not by itself apportion this gap; it is specific to this chip in the measured set. On the Radeon 780M the WGPU render point estimate is 12.2% faster, but neither it nor the 16-pass BLADE placement contrast clears that cell’s 106.7% floor; both are descriptive. The Raphael iGPU does resolve a 13.4% WGPU advantage, where the largest highlighted within-BLADE change is 1.8%, while both iGPUs are slower on compute. The discrete-against-integrated contrast is reported as measured rather than assigned to one of the bundled factors.

On the Intel device the WGPU point estimate is slower on every shared workload. Three cells have one-signed intervals, from 7.9% to 11.7%; the `graphics-chain` interval spans both signs and is not counted as a directional result. Since `compute-chain` is fully serialized in both implementations, its resolved gap cannot be an overlap difference. Given this machine’s systematic control offsets (Section 5.5), we treat even the three resolved directions as end-to-end observations rather than mechanism estimates.

Separating the candidates that remain — the memory scope of each barrier, the layout the images sit in, timestamp-window differences, or per-pass setup outside synchronization — would need a configuration this study does not have: BLADE emitting resource-scoped barriers, which is the tracking it exists to avoid. What the column does establish is that neither matched stack dominates the device side. The WGPU path wins in some cells and loses in others; because tracking, barrier form, layouts, validation, and per-pass setup move together, the end-to-end column cannot assign those signs to tracking alone. Its host record-and-submit time is higher in every measured cell (Table 5).

Workload	BLADE		WGPU	
	host	wait	host	wait
c-ind	68.3	1449.1	92.8	1449.2
c-chain	68.3	1082.3	91.1	1110.6
g-ind	85.2	1302.6	150.0	1247.9
g-chain	83.8	1421.8	149.9	1409.4

Table 7: Apple M3 (Metal), 16 passes: host cost (recording plus submission) and host wall time waiting for completion, median of process medians in microseconds. Metal GPU timestamps are reported per pass rather than as a span and were unavailable for three of the four wgpu workloads, so wall time is used instead of device time.

6.8 Apple/Metal case study

Metal is not pooled into the Vulkan comparison, because it hides image layouts and because the framework performs the hazard tracking that BLADE declines to perform on Vulkan [2]. On Metal, BLADE is not a tracking-free implementation; it is a client of Metal’s tracker. This study uses the pre-Metal-4 `MTLCommandQueue` path. Apple documents that resource hazard-tracking mode has no effect on `MTL4CommandQueue`, whose explicit queue barriers are a separate design point [2].

Table 7 therefore reports only what is comparable. BLADE’s host cost is 56–75% of WGPU’s. Completion wall time is within a few percent either way on every workload; the double-digit wait saving the pre-parity pilot appeared to show does not survive shader parity, which is consistent with it having been the injected-check artifact of Section 5.2. Wall time is not used as a substitute for device span. This recollection uses the shader-check settings of Section 5.2 on both implementations. Note that BLADE’s Metal host cost (68–85 μ s for sixteen passes) is about 1.2 times its Vulkan host cost on the RTX 5070, because each BLADE pass creates a new MTL command encoder; the Metal backend has room to improve independently of anything in this paper.

The question Metal actually poses — what hazard tracking costs when the framework rather than the application performs it — is not answered by these workloads, because both implementations pay it. A separate harness, listed in Appendix A, measures it directly by creating the same resources with `MTLHazardTrackingModeTracked` and `MTLHazardTrackingModeUntracked` and, in the untracked dependent case, reconstructing the dependency with an `MTLFence` or an `MTLEvent`. Each pass is one compute encoder with one single-thread dispatch, so the measurement is dominated by synchronization rather than by work. The full protocol and tables are in the repository’s investigation note [12].

The replicated result differs sharply from the Vulkan placement result, in the direction the single-session pilot indicated. Across 10 retained process sessions on AC power, turning tracking off for *independent* passes moves

encode time by -2.0 to $+5.3\%$ and GPU time by -3.5 to $+2.1\%$ across every session and pass count — consistent with Metal’s tracker already distinguishing the resources, and now bounded by the observed spread rather than asserted from one launch. For *dependent* passes, opting out is expensive and stable across sessions: at 100 passes an `MTLFence` costs $8.9\times$ the encode time (session range $[8.8, 9.1]$) and $10.1\times$ the GPU time of the tracker, an `MTLEvent` $8.5\times$ and $10.1\times$; at 500 passes the GPU ratios rise to $11.1\times$ and $11.1\times$.

Every session retains its raw observations, and each of the 40800 rows records the resource mode the driver actually applied and an output-correctness check; the requested mode took effect and the check passed in all of them. The harness’s deliberately unsynchronized untracked case again produced correct values, which is still not evidence that the mode is safe.

The API consequence is firmer still than the timings: `manual_barriers` should stay a Vulkan concept. Metal’s tracking mode is chosen per resource at creation, while BLADE’s flag belongs to a command encoder, and a resource outlives any one encoder; moreover BLADE’s `barrier()` is called between RAII pass encoders, after the producer’s encoder has already ended, which is too late to place the producer-side `updateFence`. The cost model is consistent with the abstraction: BLADE’s Vulkan fallback can serialize unrelated passes, whereas opting out of Metal tracking makes the application reconstruct real dependencies using an API that BLADE does not currently expose at the right lifetime.

6.9 Generated barriers and image layouts

RenderDoc captures of all six Blade workloads under three policies, and all four shared WGPU workloads, confirm what Sections 2.2 and 3.1 describe from source. The Blade counts are exactly the contract: with four passes recorded, `B-auto` emits 5 barriers per configuration—one before each pass plus the one the encoder emits on finish—`B-hazard` emits 1 on the independent workloads and 4 on the chains, and the captured `B-auto-scoped` policy emits the same number as `B-auto`. The uncaptured crossed policies follow from the same source path and are publication capture targets, not capture-backed observations in the retained artifact.

The masks match too. Every `B-auto` barrier in every workload is the same shape, `ALL_COMMANDS` to `ALL_COMMANDS`, with `TRANSFER_WRITE | MEMORY_WRITE` made available to the four reads and writes `TRANSFER_READ`, `TRANSFER_WRITE`, `MEMORY_READ`, and `MEMORY_WRITE` — including the two driver-workaround accesses. Under `B-scoped` the derivation is visible in the capture: compute-to-compute boundaries carry `COMPUTE_SHADER` to `DRAW_INDIRECT | COMPUTE_SHADER`, render-to-render carry `ALL_GRAPHICS` on both sides, mixed workloads show both crossings, the first barrier of each encoder still carries `ALL_COMMANDS` on the source

side, the barrier emitted on finish still carries it on the destination side, and the workaround accesses are absent, as intended.

Two things the Blade captures settle that source reading could not. Every steady-state Blade barrier is a global memory barrier: no buffer or image barrier appears in any Blade configuration. And no image layout transition appears in the captured measured iteration—the setup command has already moved images to `GENERAL`, and they stay there. The capture therefore observes the steady-state layout claim rather than the initialization transition itself.

The matched WGPU program was captured the same way, which directly observes barrier form and layout but does not turn the end-to-end comparison into a single-factor experiment.

Counts are equal; endpoint placement is not. WGPU emits exactly as many barrier calls as `B-hazard`: 1 on the independent workloads and 4 on the chains, where `B-auto` emits 5. The extractor replays Vulkan command buffers in `vkQueueSubmit` order and records how many draw or dispatch commands precede each call; using host recording order would be wrong because WGPU records pass and transition command buffers separately.

That pass-relative position exposes an endpoint shift. On the independent compute workload, WGPU’s one call is an initial transition (1 initial, 0 inter-pass, 0 final), whereas `B-hazard`’s one call is the encoder’s final barrier (0, 0, and 1, respectively). On the chain, both have 3 inter-pass calls, but WGPU again has an initial call and `B-hazard` a final one. Thus the tracker derives the required inter-pass placement in this workload, but it does not reproduce `B-hazard`’s complete call sequence. This also explains why equal call counts do not make their timestamp windows equivalent: WGPU’s initial transition lies before its first timestamp, while BLADE’s final barrier lies inside its reported span.

Memory scope. Every BLADE barrier is a global memory barrier and every WGPU barrier is resource-scoped — a `VkBufferMemoryBarrier` for the compute workloads, a `VkImageMemoryBarrier` for the graphics ones. Neither implementation emits a barrier of the other’s kind in any configuration. This is factor 2, which the experiment contract lists as observed-only, and it is now observed rather than read from source.

Layouts. The most useful thing the captures say is negative. WGPU’s image barriers carry `COLOR_ATTACHMENT_OPTIMAL` on *both* sides: they are execution and memory dependencies, not layout transitions. BLADE emits no image barrier at all. So neither implementation performs a steady-state layout transition on these workloads, and the layout difference between them is static — which layout the driver sees the image in — rather than a per-pass cost. Any layout effect in the end-to-end numbers is therefore a property of `GENERAL` versus `COLOR_ATTACHMENT_OPTIMAL` as states, not of moving between them.

The capture tables from all 4 machines are byte-identical — 160 rows, the same SHA-256 — which is what the design intends. These CSVs contain extracted barrier records, not the entire command stream. They license the narrower inference needed here: barrier counts, pass-relative positions, barrier kind, stages, accesses, and layouts in the relevant records do not vary by capture machine. They do not show unextracted commands or prove that every driver-internal representation is byte-identical. The retained manifests name those hosts but did not record the adapter selected by either implementation, so this is deliberately a cross-host statement, not evidence from three identified GPU models. The publication recollection pins both selectors and records the reported device name.

One case is deliberately not covered. Every workload here is single-sampled, and Section 2.2 reads from RADV that **GENERAL** disables FMASK compression unconditionally, so multisampled colour is the one configuration where a persistent **GENERAL** layout forgoes that compression on AMD. We state the predicate from driver source and do not measure its performance consequence.

The layout claim this draft makes comes from source, not a controlled layout experiment: on RADV, a persistent **GENERAL** layout retains DCC on RDNA and later under the stated queue conditions, conditionally retains HTILE, and disables FMASK compression. Our graphics workloads use single-sampled **RGBA8** colour targets: they can retain DCC and have no FMASK to lose. They contain no depth target, so the HTILE predicate is source context rather than a measured workload property. A study that wanted to stress the layout dimension would need at least MSAA colour and a depth case. The Intel system is the only one whose driver does not advertise **VK_KHR_unified_image_layouts**, and BLADE still outperforms WGPU there on both graphics workloads. That is consistent with the missing extension not dominating, but it is an end-to-end observation and not a layout experiment.

7 Discussion

The results support a decision boundary rather than a winner.

Placement is the big lever, and it does not require tracking. The largest effect among the axes controlled within BLADE comes from barrier *placement*: on the RTX 5070’s independent render passes, redundant barriers cost a third of the span (152.1 μ s for **B-auto** against 102.8 μ s for **B-hazard**). The matched WGPU path lands a couple of percentage points farther away (99.7 μ s), but that residue is end-to-end, not a measurement of tracking precision. Resource knowledge lets WGPU avoid dependencies between distinct targets automatically; an application able to say “these passes are unrelated” collects nearly all of it without tracking anything, and an

abstraction that can express neither leaves a third of this command-buffer span on the table. The counterweight is the NVIDIA render chain, where the WGPU path is 17.5% faster and no declaration BLADE accepts reaches that end-to-end result (Section 6.7).

The device decides, and reading the driver does not substitute for asking it. The captures show that all 4 machines produce identical extracted barrier tables (Section 6.9), so the cross-machine BLADE scope differences are not caused by different application barrier masks or counts. Against that fixed request, two predictions we made from mechanism failed in the same direction. Redundant barriers between independent render passes cost 32.4% on NVIDIA against 7.3% on the discrete AMD part and only 1.1% on Raphael, while the Radeon 780M sweep has threshold-cleared points in the opposite direction, despite AMD being the architecture whose vendor guidance warns hardest about broad barriers. Narrowing the barrier scope removes the most commands from the RADV stream at render-involving boundaries, where no AMD cell resolves a directional effect, and the fewest at the compute-to-compute boundaries where the discrete part actually pays: the request count and the measured effects do not correlate. Both times the source told us what would be requested and we took that for what it would cost.

The second failure had an answer in the same file, and finding it afterwards is the part worth keeping. **radv_dst_access_flush** discards the destination-side cache invalidation our narrower barrier was designed to avoid, on the measured generations whenever no render-backend-incoherent attachment is marked dirty (Section 2.2.1). We had read the function that translates a request into work and not the one that decides how much of that work to do. Counting emitted commands describes how much the driver is *asked* for; that request set can be a very loose predictor of latency, because a drain is free when there is nothing in flight to drain and an invalidation is free when the driver has already proved it unnecessary. Mechanism is good for explaining a measurement and poor for replacing one — and a mechanism that only explains what has already been measured is still worth having, because it says where the result should generalize. Here it says: on a part whose TCC is not coherent with the render backends, an AMD render-side saving is a concrete hypothesis. We did not have one to try.

The workload envelope for coarse barriers. Tracking-free synchronization with pass-boundary barriers is a candidate when passes form chains, when each pass leaves little device headroom, or when host cost dominates. It becomes risky when independent passes have complementary bottlenecks and spare device capacity, which is where the field report sits. An encoder-level opt-out covers that case at negligible API cost, which is why it was the right resolution to the field report [7]: it turns the expensive case into a two-line change without adding resource states to the API.

The end-to-end host difference is the most consistent result. The matched WGPU stack takes more record-and-submit time in every fixed-matrix cell across four vendors, and the timestamp-free RTX 5070 sweep confirms the difference over the measured pass counts. The fixed-matrix ratios (1.3–5.9 $\times$) include timestamp instrumentation, and the sweep becomes non-linear at larger counts, so we do not extrapolate them to hundreds of passes or whole-frame savings.

What tracking itself costs the CPU is bracketed, not measured. The question this study set out from — how much host time WGPU sacrifices specifically for resource tracking — gets a lower and an upper bound here, and they are far apart. Symbols directly attributable to the trackers are 1–19% of WGPU’s whole-process samples, a bound blind to inlined tracker work; the full end-to-end gap is the upper bound, and it bundles validation, lifetime management, bind-group resolution, and command translation with the trackers — machinery that exists partly *because* state is tracked but is not the tracking itself. The experiment that would separate them — a WGPU build with tracking short-circuited, or a profile gated to the record-and-submit interval on a pinned adapter (Section 6.6) — was not performed. Anyone quoting this paper as “tracking costs $N\times$ ” is quoting a bracket as a point.

The negative results matter. The Radeon 780M sweep contains threshold-cleared regressions, showing that removing redundant barriers is not a free optimization, and the AMD render boundaries show that a saving visible in a command stream need not be visible in a timer. A layer that exposes manual barriers must therefore treat them as a measured optimization rather than a default, and profiling guidance belongs next to the API.

7.1 Implications for engine architecture

The architectural result concerns where resource knowledge lives, not whether it exists. An engine that already has a render or task graph need not reproduce that graph as per-resource state inside its RHI. It can use the graph to group independent producers, place a BLADE barrier at a dependency cut, and then record the consumers. The field report has exactly this shape: two independent views form one phase and join before refraction. In a phase-structured graph, one global barrier can represent many aligned resource edges.

A global barrier is nevertheless a *cut*, not an edge. Suppose one producer must complete before a consumer while another long producer is independent of that consumer. A global barrier placed for the first dependency also constrains the second producer whenever it falls on the barrier’s source side and matches its scope; a resource-specific barrier can name only the real edge. The controlled workloads here are the two endpoints — all inter-pass edges or none — plus a mixed chain whose incoming passes are almost all hazardous. They do not show that an arbitrary partially dependent render graph can be lowered to

global cuts without losing scheduling freedom.

For a small or immediate-mode renderer without a graph, automatic pass boundaries remain the conservative starting point. Deriving stage and access scope from pass kind is internal, requires no resource declarations, and caused no resolved regression in the measured cells. Manual placement is then a profiler-guided escape hatch for a region where unrelated passes demonstrably serialize; it is not a new default for the whole frame.

Finally, a tracking-free RHI is not a resource-management system. Transient aliasing and lifetime planning, subresource layout transitions, queue-family ownership and cross-queue dependencies, and the multisampled-colour exception identified on RADV all still require resource identity somewhere. An engine-level graph can own those responsibilities; an engine that wants the RHI to infer them needs a tracked abstraction such as WGPU or NVRHI. The choice exposed by this study is whether that knowledge must be duplicated on the command-recording path when the engine already has it.

7.2 What to do on AMD

The short answer to “does this abstraction work on AMD” is that its host-side advantage holds there as firmly as anywhere — 56.2 μ s against 10.6 μ s for the same sixteen passes on the RX 7900 XT — and its device-side cost is the widest-ranging of any vendor in the study, from 32.3% recoverable to 42.0% lost by trying. Three things follow, in the order an application should try them.

Take the narrow barrier, and expect it to pay only between compute passes. It is tempting to read Section 2.2 as a shopping list of savings, and we did. What the measurement licenses is narrower: a 6.7% [4.5, 6.9] reduction at this 16-pass shape on the discrete part where a compute pass hands to a compute pass, with no directional effect meeting the exploratory criterion at render-involving boundaries or on the Radeon 780M; Raphael’s two compute cells show only the threshold-edge 0.2% observation discussed in Section 6.3. Section 2.2.1 gives a conditional account of why much of the render-side request may already be elided. No regression clears the stability diagnostic in these tests; because the scope is derived internally rather than asserted by the application, it is the lower-risk change to take first. It is not an established AMD render optimization, and the practically meaningful saving observed here appears only under automatic placement (below).

Restructure the passes so the remaining barriers are ones you need. This is the change that pays. On the RX 7900 XT, sixteen independent compute passes run 32.3% [32.1, 32.6] faster once the redundant barriers are gone, and the mixed equivalent 17.7% faster. The field report’s renderer serialized because independent work sat on either side of a barrier. The fix is not only to delete that barrier but to group the independent passes so one barrier separates each group from the next — ray trac-

ing with the reflection prepass, both view passes together, then the resolves, then the dependent refraction pass. With `manual_barriers` this is a change to where `barrier()` is called, not a change to any resource declaration, and the encoder-level flag makes the scope of the change explicit. Applications that want the automatic policy for most of a frame can put the restructured part in its own encoder and submit both to the same queue.

Then measure, because the restructuring is not always a win. On the Radeon 780M, the 16-pass mixed-independent cell costs 25.7% [24.8, 57.4] against a 23.9% floor. The corresponding graphics-independent point is 42.0% but is not resolved because launch-to-launch dispersion raises the floor; the graphics sweep resolves the same direction at 2, 32, and 64 passes, with descriptive medians suggesting growth and saturation (Figure 2). On the RX 7900 XT the same change on render passes instead bought 7.3% — real, but a quarter of the 32.3% its compute equivalent buys. Two AMD parts of the same generation therefore disagree about the sign on render work, which is the strongest reason in this paper not to ship the change unmeasured. The passes worth grouping are the ones with complementary bottlenecks — ray tracing next to rasterization, a depth prepass next to a compute simulation — not several instances of the same bottleneck. A GPU profiler showing the two passes actually overlapping, as in the field report, is the evidence to look for; a frame-time delta on its own will not distinguish a scheduling win from whatever the 780M is doing.

Note also that suppressing the encoder’s barriers forfeits the scope narrowing on the boundaries that remain, since an explicitly placed barrier describes only its source. On AMD that now costs the compute-boundary saving; on NVIDIA it is a real trade everywhere the scope pays.

Two things we looked for and did not find are worth recording, because they bound the mechanisms examined here. The narrowed scope removes the operations Section 2.2 identifies without naming a resource, and on the discrete AMD device that changes compute boundaries by a fraction of a microsecond. The remaining pessimism that a global barrier requests in RADV — including metadata-flush requests and the unavailable image-specific coherence check — is only avoidable by supplying an image, which is the state BLADE declines to keep. The remaining AMD lever is placement, and placement is the application’s knowledge, not the layer’s.

The one case where AMD still argues for genuine layout tracking is multisampled colour, where RADV disables FMASK compression in `GENERAL`. That is orthogonal to everything above and is not addressed by either barrier axis.

8 Threats to Validity

GPU timestamps could perturb the host measurement, and the paired RTX 5070 sweeps test for it: their times-

tamped and timestamp-free host medians agree at every pass count, so on that machine the fixed matrix’s host columns carry no measurable instrumentation inflation. The other machines have no such pair, so the concern is tested on one driver and assumed bounded, not proven absent, on the rest. Device-side BLADE-WGPU comparisons also use different timestamp counts and endpoints (17 writes including Blade’s final barrier versus two writes ending with wgpu’s last pass), so they remain end-to-end instrumented spans; within-BLADE comparisons use identical instrumentation.

Driver heuristics may recognize repeated synthetic work; the benchmark varies a per-iteration seed while preserving the graph shape, but does not vary resource identity. Results from one driver version cannot establish permanent vendor behaviour. The local collection directories retain metadata for comparison, but they are git-ignored; submission therefore requires a checksummed raw-data artifact rather than a claim that the repository alone reproduces the tables.

The controlled BLADE variants now isolate two of the four factors — placement and stage/access scope — but still not global versus resource-specific memory scope, nor persistent `GENERAL` versus usage-specific layouts. Those two are argued from driver sources and observed end-to-end through WGPU, so causal claims about them are limited by construction. The scoped results cover five Vulkan device-driver pairs across three vendors, but not every cell within them: 10 of 30 have control floors above 2%, and every directional conclusion is checked against the post-hoc stability threshold of the cell and pass count it comes from; curve shapes and state-conditioned contrasts are labelled descriptive. Every archival timing collection is preceded by a retained synchronization-validation matrix, run with the Khronos checks forced on and its logs stored beside the timing rows; the timing rows themselves come from the validated shaders and layered output checks of Section 5.2. Retained validation is still evidence about the schedules that were executed rather than a proof. The automated workload matrix exercises compute, render, and their crossings; it does not exercise the transfer or acceleration-structure rows of the pass-kind table. Those rows have a source-level correctness argument but need dedicated synchronization-validation coverage before the scoped mode is presented as validated for every BLADE pass kind. End-to-end WGPU differences are labelled as such throughout, and the ones we cannot account for — the Intel gap in one direction, the RX 7900 XT gap in the other — are left unexplained rather than attributed.

Each configuration has 30 within-process samples and ten independent process-level repetitions. The paired hierarchical bootstrap preserves that nesting and reveals variation a pooled analysis hides — at three pilot blocks it disqualified cells that ten now resolve, and on one host it exposes launch-to-launch dispersion rather than averaging it away. Ten blocks still estimate tails coarsely, the inner resampling treats time-ordered observations as

exchangeable, and the matrix has no second-day or in-process-alternation sensitivity run. A moving-block or process-median-only re-analysis of the archived observations remains a worthwhile check.

The workloads are microbenchmarks of sixteen passes each. Two of the six alternate compute and render work, so heterogeneity is represented, but only the synthetic kind: two pass types in a fixed alternation, with identical work in every instance of each. Real frames vary the cost, the resources, and the bottleneck from pass to pass, which is precisely the regime in which the field report saw a benefit and our **graphics-independent** workload saw a loss on one device. The boundary proposed in Section 7 is therefore a hypothesis supported by microbenchmarks — including mixed ones — and one field report, not a validated rule. We chose not to close that gap with application workloads: doing so would trade a controlled comparison for a plausible one, since scenes, content, and driver heuristics vary in ways this design exists to hold still. A reader who needs the rule validated on their own frame should measure their own frame, which is the recommendation Section 7.2 makes anyway.

The dependency structures are also endpoints rather than representative DAGs. The independent workloads remove every timed inter-pass barrier, while the single-kind chains retain every one; in **mixed-chain**, every incoming pass from the third onward has a dependency and the policy is intentionally conservative at the second pass. No controlled workload contains two interleaved chains whose dependency edges cross different global cuts. Such a staggered partial DAG is the most important next microbenchmark for the engine claim: it would measure the scheduling freedom lost when one real edge forces a global cut across otherwise independent work.

The AMD evidence is now a discrete RDNA3 part, a mobile RDNA3 iGPU, a very small desktop iGPU, the driver sources, and a user report. The small iGPU is the most precisely measured device in the study and the least responsive to barrier changes, consistent with but not proving single-pass saturation. The pilot’s version of this paragraph asked for the discrete part’s placement cells to be repeated with enough warm-up for the device to stop accelerating before being leaned on — its clocks were already pinned, and that was not enough. This revision is that repeat, and the claims survived it sharpened rather than softened: the compute effect grew while its floor fell to 0.5%, the render cell went from unresolvable to a resolved 7.3%, and the scope axis, unreadable on the pilot, resolved to 6.7% at discrete compute boundaries, with only threshold-edge 0.2% observations on Raphael and no render-boundary result. The third AMD part still disagrees in sign on the render workloads, which is itself part of the result rather than a contradiction of it.

Finally, no mobile tiler is represented at all. A pass-boundary barrier policy would be stressed hardest exactly there — a barrier that breaks a render pass on a tile-based GPU forces a tile flush and reload — so the class where

this design has the most to lose is the class the study says nothing about.

9 Related Work

The Vulkan specification defines explicit execution and memory dependencies and application-controlled image layouts [8]. AMD and NVIDIA have published architecture-specific synchronization guidance [4, 1, 19]. The unified-image-layouts extension standardizes the observation that specialized layouts no longer correspond to distinct representations on many modern implementations [23]; the RADV predicates quoted in this paper are a concrete instance of that convergence, including its exceptions.

Direct3D 12 Enhanced Barriers separate work synchronization, memory access, and texture layout, and provide global as well as resource-specific barrier types [9]. This closely matches the factorization used here. The present study differs by evaluating an RHI policy that chooses global barriers systematically at pass boundaries and by measuring that policy through Vulkan; it makes no Direct3D performance claim.

Render graphs take the workload declaration as their source of resource lifetimes and pass dependencies. Frostbite’s FrameGraph is a canonical production example: it builds a graph of render passes and resources so the engine can derive scheduling and resource-management decisions centrally [17]. Activision’s Task Graph reports the maintenance failures that motivated the same move and its scale in a shipping renderer [3]; Unreal’s RDG is a current production system deriving transitions, queue fences, aliasing, and validation from pass parameters [6]. These systems are not alternatives to BLADE. An upstream graph may select BLADE’s dependency cuts. What BLADE declines is the resource declaration *inside the RHI*, and its smaller contract therefore cannot itself derive resource-scoped barriers, aliasing, or queue schedules.

Academic rendering systems explore the resource-aware end of the same design space. LuisaRender builds a subresource dependency graph and reorders independent commands, reporting up to a 19% rendering-time improvement from its command scheduler [24]. RenderKernel moves resource declarations into a compile-time DSL and generates Vulkan command recording and barrier insertion [22]. Both demonstrate the value of rich resource knowledge in a framework or compiler. BLADE does not dispute that value; it asks whether a low-level RHI must maintain the same knowledge at run time when an upstream system already owns it.

WebGPU specifies portable usage-scope validation [20]; WGPU implements it using optimized dense resource trackers [21]. NVRHI is the closest prior RHI design point: it tracks resource and optional subresource state per command list, while allowing automatic barriers to

be disabled for lower-frequency explicit state calls when an application wants to reduce CPU overhead or control placement [18]. Unlike BLADE, that manual mode retains the per-resource state machinery and its explicit calls still name resources. Metal takes a third position, performing hazard tracking in the framework with an opt-out per resource for `MTLCommandQueue`; Metal 4 queue barriers are outside this study [2]. BLADE occupies the tracking-free RHI point by exposing an unsafe native contract and relying on application-declared pass dependencies.

10 Artifact

The manuscript, collectors, and BLADE implementation are public on the `blade-sync-study` branch, and the matched program is public on the corresponding WGPU fork branch [13, 14]. The archival round measures the corrected benchmark on every machine: WGPU revision `7d37a77` and BLADE revision `87ed067`, with the study host’s sweeps at one later commit that changed only the paper directory and therefore did not change the compiled benchmark. The bibliography records the full identifiers, and per-collection manifests record the revisions, the clock configuration, and device metadata.

The measurements are the irreplaceable half, and they travel with this submission: the ancillary files hold every raw collection — timing matrices, sweeps, retained synchronization-validation logs, host profiles, captures with extracted barrier tables, and the Metal hazard sessions — with a digest of every file. The measured code is public and content-addressed: both repositories carry study-code tags (`sync-study-v1` on BLADE, `blade-sync-study-v1` on the WGPU fork), and the commit hashes recorded in the bibliography and in every collection manifest remain the identifiers of record — verifiable from any clone or archive of either repository. The tags pin the code, not this article revision; arXiv preserves the submitted article source. Extracting the collections into `paper/data/raw/` in a study-branch checkout containing this submission’s analysis tooling and running the table builder regenerates every table, figure, and quoted number in this paper; a missing collection is a build error rather than a silent absence.

Generative-AI disclosure. Anthropic Claude and OpenAI Codex were used interactively for literature discovery, manuscript editing, and development and review of benchmark and analysis tooling. They are not authors; the human author takes responsibility for the cited sources, code, analyses, and final text.

11 Conclusion

This study does not isolate a single price for fine-grained resource state: the WGPU comparison also changes validation, barrier form, layouts, and command machinery.

Within BLADE, it does isolate the effect of placement for the measured cells. The dependent-chain variants have the same timed barrier calls by construction and no chain interval clears its cell-specific stability threshold; removing fifteen redundant inter-pass barriers reduces device span by 29.3% and 32.3% on the two discrete GPUs, while the Radeon 780M graphics sweep resolves a roughly two-fifths penalty at 32 and 64 passes, for reasons we could not establish. Its 16-pass graphics point has the same magnitude but does not clear that noisy cell’s control floor; the 16-pass mixed-independent regression does. The matched WGPU stack takes more record-and-submit time in every measured cell, but that end-to-end difference must not be called the cost of tracking alone.

The corresponding BLADE development outcome is deliberately small. The mainline Vulkan encoder can retain its existing API and add one encoder-wide pass-kind summary: a bitmask of producers recorded since the previous barrier. An automatic boundary can then derive both sides of its global barrier from the accumulated producers and the pass being opened; an explicitly placed barrier derives its source and keeps a conservative destination. This is lightweight aggregate access tracking, not per-resource state: it stores no resource identities, subresources, layouts, or lifetimes. The AMD measurements also make this a scope refinement rather than a vendor heuristic. Narrowing paid at the discrete part’s compute boundaries but not at its render boundaries, while removing barriers from independent render work hurt the Radeon 780M. BLADE therefore should neither select placement by vendor nor assume that fewer barriers are always faster.

The practical conclusion is that resource-dependency knowledge is indispensable, but per-resource state inside the RHI is not always. For the phase-structured field report and the independent-pass endpoints measured here, the expressive gap that matters is the ability to say “these passes are unrelated”. An engine-level graph can remain the source of truth and drive BLADE’s dependency cuts without maintaining a second tracker in the abstraction. An encoder-level opt-out gives an application that already knows its dependency graph nearly the same device-span reduction that the matched WGPU path shows on the RTX 5070 independent-render cell, while retaining BLADE’s lower measured host cost, without adding per-resource state surface. That is a cell-specific end-to-end comparison, not a decomposition of tracking cost, and the current workloads do not establish the same result for a partially dependent DAG. The opt-out does add an encoder-level mode and explicit barrier calls. What the current call cannot reach is a destination scope derived from a future pass, and, on one NVIDIA render chain, a further 17.5% bundled among barrier form, layout, timestamp window, and other end-to-end differences. A global barrier can express a narrow destination mask; BLADE’s current explicitly placed call lacks the information to derive one automatically. The opt-out should be presented as a profiled optimization rather than a default, because

on at least one real device it makes things worse.

The remaining bounds are specific: no application frames or multisampled targets, no mechanism for the Radeon 780M overlap penalty or the discrete AMD scope saving, and no controlled layout variant. They bound the corresponding claims rather than supporting extrapolation beyond the measured workloads.

References

- [1] AMD. RDNA performance guide. <https://gpuopen.com/learn/rdna-performance-guide/>, 2023. Updated for RDNA 3; accessed 2026-07-24.
- [2] Apple. Resource synchronization. <https://developer.apple.com/documentation/metal/resource-synchronization>, 2026. Includes pre-Metal-4 hazard tracking and Metal 4 queue synchronization; accessed 2026-07-28.
- [3] Charlie Birtwistle and François Durand. Task graph renderer at Activision. https://enginearchitecture.org/downloads/reac2023_task_graph_renderer.pdf, 2023. Rendering Engine Architecture Conference 2023; accessed 2026-07-28.
- [4] Matthäus Chajdas. Vulkan barriers explained. <https://gpuopen.com/learn/vulkan-barriers-explained/>, 2016. AMD GPUOpen; accessed 2026-07-24.
- [5] A. C. Davison and D. V. Hinkley. *Bootstrap Methods and their Application*. Cambridge University Press, 1997. <https://doi.org/10.1017/CB09780511802843>.
- [6] Epic Games. Render dependency graph in Unreal Engine. <https://dev.epicgames.com/documentation/en-us/unreal-engine/render-dependency-graph-in-unreal-engine>, 2026. Unreal Engine 5.8 documentation; accessed 2026-07-28.
- [7] EriKWDev and Dzmitry Malyshau. Proposal: groups of commands with a shared barrier instead of forced always? Or manual barriers? <https://github.com/kvark/blade/issues/343>, 2026. Blade issue 343, opened 16 April 2026, closed 10 June 2026.
- [8] Khronos Vulkan Working Group. Vulkan specification. <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>, 2026. Accessed 2026-07-24.
- [9] Bill Kristiansen. D3D12 enhanced barriers preview. <https://devblogs.microsoft.com/directx/d3d12-enhanced-barriers-preview/>, 2021. Microsoft DirectX Developer Blog; accessed 2026-07-28.
- [10] Adam Loy and Jenna Korobova. Bootstrapping clustered data in R using lmeresampler. *The R Journal*, 14(4):103–120, 2023. <https://doi.org/10.32614/RJ-2023-015>.
- [11] Dzmitry Malyshau. Add manual_barriers flag to commandencoderdesc. <https://github.com/kvark/blade/pull/355>, 2026. Blade pull request 355, merged 10 June 2026.
- [12] Dzmitry Malyshau. Metal untracked hazard mode on Apple M3. <https://github.com/kvark/blade/blob/87ed06750877f336ad1c54fa5005a0b799f488d7/docs/metal-hazard-tracking.md>, 2026. Investigation of 24 July 2026 at Blade revision ba0fb5a; note frozen at study revision 87ed06750877.
- [13] Dzmitry Malyshau and contributors. Blade. <https://github.com/kvark/blade/tree/blade-sync-study>, 2026. Study branch; the study-code snapshot is tagged sync-study-v1, and every collection manifest in the ancillary data records the exact measured revision.
- [14] Dzmitry Malyshau et al. wgpu synchronization-study benchmark fork. <https://github.com/kvark/wgpu/tree/blade-sync-study>, 2026. Benchmark at revision 7d37a77c086f3d3b8a9bdba6b476cd7ac195fcfc, tagged blade-sync-study-v1.
- [15] Mesa contributors. RADV: radv_cmd_buffer.c and radv_image.c. <https://gitlab.freedesktop.org/mesa/mesa/-/tree/d18d598e275d/src/amd/vulkan>, 2026. Read at Mesa main revision d18d598e275d, 25 July 2026.
- [16] Mesa contributors. Vulkan runtime: vk_synchronization.c. https://gitlab.freedesktop.org/mesa/mesa/-/blob/d18d598e275d/src/vulkan/runtime/vk_synchronization.c, 2026. Read at Mesa main revision d18d598e275d, 25 July 2026.
- [17] Yuriy O’Donnell. FrameGraph: Extensible rendering architecture in Frostbite. <https://www.gdcvault.com/play/1024612/FrameGraph->, 2017. Game Developers Conference 2017; accessed 2026-07-27.
- [18] Alexey Panteleev. Writing portable rendering code with NVRHI. <https://developer.nvidia.com/blog/writing-portable-rendering-code-with-nvrhi/>, 2021. NVIDIA Technical Blog; accessed 2026-07-28.

- [19] Nuno Subtil, Matthew Rusch, and Ivan Fedorov. Tips and tricks: Vulkan dos and don'ts. <https://developer.nvidia.com/blog/vulkan-dos-donts/>, 2019. Updated 14 January 2025; accessed 2026-07-24.
- [20] W3C GPU for the Web Working Group. WebGPU. <https://gpuweb.github.io/gpuweb/>, 2026. Editor's Draft, 27 July 2026, source revision d390da5f80f18e82d9535a40c6f2f1f65e6884ae; accessed 2026-07-28.
- [21] wgpu contributors. Resource state and lifetime trackers. https://docs.rs/wgpu-core/30.0.0/src/wgpu_core/track/mod.rs.html, 2026. wgpu-core 30.0.0 source documentation; accessed 2026-07-24.
- [22] Jinyuan Yang, Soumyabrata Dev, and Abraham G. Campbell. RenderKernel: High-level programming for real-time rendering systems. *Visual Informatics*, 8:82–95, 2024. <https://doi.org/10.1016/j.visinf.2024.09.004>.
- [23] Shahbaz Youssefi et al. VK_KHR_unified_image_layouts extension proposal. https://docs.vulkan.org/features/latest/features/proposals/VK_KHR_unified_image_layouts.html, 2024. Revision 1; accessed 2026-07-24.
- [24] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. LuisaRender: A high-performance rendering framework with layered and unified interfaces on stream architectures. *ACM Transactions on Graphics*, 41(6):232:1–232:19, 2022. Author manuscript: <https://www.cs.ucr.edu/~ygu/papers/SIGASIA22/LuisaRender.pdf>.

increments its own four-byte buffer, and a *dependent* one in which every pass increments the same buffer. In the dependent untracked cases the harness reconstructs the ordering explicitly, with either one `MTLFence` updated and waited on between consecutive encoders or one `MTLEvent` signalled and waited on between them.

Resources and synchronization objects are reused across iterations. Each case receives ten warm-up runs, cases are interleaved in alternating order to reduce mode-order and thermal bias, and results are medians of 200 samples, except the 500-pass cases which use 80. Encoding time excludes `commit()`, which is measured separately; device time comes from `gpuStartTime` and `gpuEndTime`. The archival run repeats the whole procedure as 10 separate process sessions on AC power, driven by a collection script distributed with the artifact; every session retains its raw per-iteration observations, the resource mode the driver actually applied, and an output-correctness check per row. The harness also contains a deliberately unsynchronized untracked case as a canary; it produced correct values on this machine and workload, which is not evidence that the mode is safe.

A Metal tracked-versus-untracked harness

The Metal question in Section 6.8 — what hazard tracking costs when the framework rather than the application performs it — is not answered by the main workloads, because both implementations pay it there. A standalone Swift harness measures it and is distributed with the artifact.

It allocates the same resources twice, once with `MTLHazardTrackingModeTracked` and once with `MTLHazardTrackingModeUntracked`, and queries each buffer's `hazardTrackingMode` afterwards to confirm the requested mode took effect. Each pass is one compute encoder containing one single-thread dispatch, so the measurement is dominated by synchronization rather than by work. Two workloads are run at 1, 10, 100, and 500 passes: an *independent* one in which every pass